

UNIVERSITÉ CATHOLIQUE DE L'AFRIQUE DE L'OUEST

Unité Universitaire du Togo (UCAO UUT)

École Supérieure d'Ingénieurs

Master BIG DATA IA

MÉMOIRE DE FIN D'ÉTUDES

Titre :

Pipeline Big Data de Monitoring de la Désinformation
en Temps Réel avec Détection du Concept Drift

Présenté et soutenu par :

KOMOSSI Sosso

Sous la direction de :

Mr TCHANTCHO Leri, Enseignant-Chercheur en Big Data & IA

Mr BABA Kpatcha, Enseignant-Chercheur en Intelligence Artificielle

Année académique : 2025 – 2026

Date de soutenance : [Mois] 2026

Remerciements

La réalisation de ce mémoire de fin d'études n'aurait pas été possible sans le concours précieux de nombreuses personnes à qui je souhaite exprimer toute ma reconnaissance.

En premier lieu, j'adresse mes plus vifs remerciements à mon directeur de mémoire, Mr TCHANTCHO Leri et Mr BABA Kpatcha, mes deux co-directeurs de mémoire,, pour sa disponibilité constante, ses orientations scientifiques éclairées et la rigueur intellectuelle qu'il m'a transmise tout au long de ce travail. Sa connaissance approfondie du domaine du Big Data et de l'Intelligence Artificielle a été un guide précieux dans l'élaboration de cette étude.

Je tiens également à exprimer ma gratitude à l'ensemble du corps professoral de l'École Supérieure du Numérique et des Technologies Avancées de l'UCAO UUT, dont les enseignements de qualité ont constitué le socle théorique et pratique indispensable à ce projet. Je remercie particulièrement les enseignants du programme de Master en BIG DATA IA pour leur pédagogie et leur engagement.

Mes remerciements s'adressent également à la direction de l'UCAO UUT pour avoir mis à disposition les infrastructures nécessaires au développement de ce projet, notamment les ressources informatiques et l'accès aux bases de données académiques.

Je suis reconnaissant(e) envers la communauté open source — les équipes d'Apache Kafka, Apache Spark, HuggingFace, River et Docker — dont les outils gratuits et performants ont rendu possible la démo technique présentée dans ce mémoire.

Je remercie chaleureusement mes camarades de promotion pour les échanges fructueux, les séances de travail collaboratif et le soutien moral tout au long de cette aventure académique.

Enfin, je dédie ce mémoire à ma famille, pour son soutien indéfectible, sa patience et ses encouragements qui ont été une source d'énergie inestimable. À mes parents, qui ont toujours cru en moi et m'ont donné les moyens d'atteindre mes ambitions — cette réussite est aussi la vôtre.

Résumé

La prolifération de la désinformation sur les médias sociaux et les plateformes d'information en ligne constitue l'une des menaces les plus sérieuses pour la cohésion sociale et la démocratie à l'ère numérique. Les approches classiques de détection, fondées sur des modèles entraînés une seule fois (modèles statiques ou batch), se révèlent structurellement inadaptées à l'évolution permanente des narratifs, du vocabulaire et des tactiques de désinformation. Ce mémoire présente la conception, le développement et la validation expérimentale d'un pipeline Big Data entièrement dynamique dédié au monitoring en temps réel de la désinformation, intégrant un apprentissage en ligne continu (online learning) couplé à un mécanisme de détection du Concept Drift pour identifier statistiquement le moment où une rumeur diverge de la réalité et commence à se viraliser.

L'originalité centrale de ce travail réside dans son approche dynamique : contrairement aux systèmes existants qui attendent la détection d'une dérive pour déclencher un réentraînement batch, notre pipeline implémente un apprentissage incrémental continu via des mises à jour de gradient sur chaque micro-batch Spark (2 secondes), combiné à un mécanisme de mémoire épisodique par réservoir sampling pour prévenir l'oubli catastrophique. Le modèle NLP, un DistilBERT à apprentissage continu (Continual-DistilBERT) pré-entraîné sur un corpus de ~153 064 exemples (après fusion et déduplication des 4 datasets disponibles) — agrégant quatre datasets : ISOT (44 898 articles), WELFake (72 134 articles), LIAR (12 836 déclarations) et FakeNewsNet (23 196 articles) — se met à jour en continu à chaque nouveau flux de données. La détection du Concept Drift (ADWIN, PageHinkley, KSWIN) agit en couche de surveillance orthogonale, signalant les ruptures de distribution et modulant dynamiquement le taux d'apprentissage.

L'architecture repose sur une stack open source de pointe : Apache Kafka 3.7 pour l'ingestion multi-sources, Apache Spark 3.5 Structured Streaming pour le traitement distribué en micro-batches, River 0.21 pour les algorithmes de drift, et Grafana 10 et Streamlit 1.38 pour le tableau de bord et l'interface interactive temps réel. Les résultats expérimentaux démontrent qu'un F1-score de 93.2 % est atteint après convergence du modèle dynamique sur le corpus complet, dépassant de 2.1 points le meilleur modèle statique comparable. Sur des scénarios de dérive simulant l'émergence d'une campagne de désinformation virale, le modèle dynamique maintient un F1-score supérieur de 15 points à celui du modèle statique à 2 000 messages post-dérive, avec une latence de bout en bout inférieure à 2,5 secondes pour un débit de 500 articles/seconde.

Ce travail constitue, à notre connaissance, la première implémentation open source d'un pipeline Big Data combinant apprentissage NLP en ligne continu, détection multi-algorithmes du Concept Drift et réponse adaptative dynamique sur des flux de désinformation, entièrement déployable via Docker Compose.

Mots-clés : Big Data, Désinformation, Fake News, Concept Drift, ADWIN, KSWIN, Online Learning, Apprentissage Continu, Reservoir Sampling, Apache Kafka, Spark Structured Streaming, Continual-DistilBERT, NLP, Temps Réel, Monitoring, Viralisé des rumeurs, Fakeddit, WELFake.

Abstract

The proliferation of disinformation on social media and online news platforms represents one of the most serious threats to social cohesion and democracy in the digital age. Classical detection approaches, based on statically trained models, are structurally inadequate for coping with the permanent evolution of disinformation narratives, vocabulary, and tactics. This thesis presents the design, development and experimental validation of a fully dynamic Big Data pipeline dedicated to real-time disinformation monitoring, integrating continuous online learning combined with a Concept Drift detection mechanism to statistically identify the moment a rumor diverges from reality and begins to go viral.

The central originality of this work lies in its dynamic approach: unlike existing systems that wait for drift detection to trigger batch retraining, our pipeline implements continuous incremental learning via gradient updates on each 2-second Spark micro-batch, combined with an episodic memory mechanism (reservoir sampling) to prevent catastrophic forgetting. The NLP model — a Continual-DistilBERT pre-trained on a corpus of ~153,064 examples aggregating four public datasets (ISOT: 44,898 articles; WELFake: 72,134 articles; FakeNewsNet: ~23,196 articles; LIAR: 12,836 statements) — Fakeddit was removed from the project (inaccessible from all sources in 2025) — updates continuously with each new data batch. Concept Drift detection (ADWIN, PageHinkley, KSWIN) acts as an orthogonal monitoring layer, signaling distribution breaks and dynamically modulating the learning rate.

Experimental results demonstrate a F1-score of 93.2% after convergence of the dynamic model on the full corpus, outperforming the best comparable static model by 2.1 percentage points. Under drift scenarios simulating the emergence of a viral

disinformation campaign, the dynamic model maintains a F1-score 15 points higher than the static model at 2,000 messages post-drift, with end-to-end latency under 2.5 seconds at 500 articles/second throughput.

Keywords: Big Data, Disinformation, Fake News, Concept Drift, ADWIN, KSWIN, Online Learning, Continual Learning, Reservoir Sampling, Apache Kafka, Spark Structured Streaming, Continual-DistilBERT, NLP, Real-Time, Monitoring, Rumor Virality, Fakeddit, WELFake.

Table des Matières

Remerciements3

Résumé / Abstract4

Table des Matières6

Liste des Figures8

Liste des Tableaux9

Liste des Abréviations10

1. INTRODUCTION GÉNÉRALE11

- 1.1 Contexte et enjeux11
- 1.2 Problématique13
- 1.3 Objectifs et résultats attendus14
- 1.4 Méthodologie générale16
- 1.5 Structure du document17

2. REVUE DE LA LITTÉRATURE18

- 2.1 Introduction à la littérature18
- 2.2 Concepts clés19
- 2.3 Analyse des études précédentes24
- 2.4 Identification des lacunes28

3. MÉTHODOLOGIE30

- 3.1 Présentation des données30
- 3.2 Prétraitement des données33
- 3.3 Architecture du pipeline35
- 3.4 Modèles et algorithmes39
- 3.5 Validation47
- 3.6 Outils et technologies49

4. RÉSULTATS52

- 4.1 Présentation des résultats52
- 4.2 Visualisation55
- 4.3 Interprétation57
- 4.4 Comparaison avec les hypothèses59

5. DISCUSSION61

- 5.1 Analyse critique61
- 5.2 Comparaison avec la littérature63
- 5.3 Limites de l'étude65
- 5.4 Implications pratiques66
- 5.5 Recommandations pour les travaux futurs68

6. CONCLUSION70

Bibliographie72

Annexes78

Annexe A — Code source : Producteur Kafka (scraper RSS)78

Annexe B — Code source : Consommateur Spark Structured Streaming80

Annexe C — Code source : Module de détection du Concept Drift (ADWIN)⁸²

Annexe D — Code source : Fine-tuning DistilBERT⁸⁴

Annexe E — Docker Compose complet du pipeline⁸⁶

Annexe F — Configuration Grafana Dashboard⁸⁸

Liste des Figures

- Figure 1.1 — Vue d'ensemble du pipeline Big Data de monitoring de la désinformation12
- Figure 1.2 — Cycle de vie d'une rumeur virale sur les réseaux sociaux13
- Figure 2.1 — Taxonomie des types de Concept Drift (abrupt, graduel, incrémental, récurrent)20
- Figure 2.2 — Architecture Lambda vs Architecture Kappa pour le traitement de flux21
- Figure 2.3 — Fenêtre adaptative ADWIN : détection d'une dérive abrupte23
- Figure 2.4 — Pipeline NLP générique pour la détection de fake news26
- Figure 3.1 — Architecture détaillée du pipeline Big Data proposé36
- Figure 3.2 — Flux de traitement de l'ingestion à l'alerte37
- Figure 3.3 — Structure du modèle DistilBERT fine-tuné40
- Figure 3.4 — Algorithme ADWIN — fenêtre glissante adaptative43
- Figure 3.5 — Protocole PageHinkley — zones d'avertissement et de dérive45
- Figure 3.6 — Stratégie de validation : sequential evaluation + holdout48
- Figure 4.1 — Courbe d'apprentissage du modèle DistilBERT53
- Figure 4.2 — Matrice de confusion — Classification statique53
- Figure 4.3 — Évolution du F1-score en temps réel avec déclenchement ADWIN55
- Figure 4.4 — Tableau de bord Grafana — Vue principale56
- Figure 4.5 — Comparaison des métriques de performance : DDM vs PageHinkley vs ADWIN58

Liste des Tableaux

- Tableau 2.1 — Comparaison des algorithmes de détection du Concept Drift²²
- Tableau 2.2 — Benchmarks des modèles NLP pour la détection de fake news²⁷
- Tableau 2.3 — Comparaison des frameworks de stream processing²⁹
- Tableau 3.1 — Caractéristiques du dataset FakeNewsNet³¹
- Tableau 3.2 — Caractéristiques du dataset GDELT (flux temps réel)³²
- Tableau 3.3 — Hyperparamètres du fine-tuning DistilBERT⁴²
- Tableau 3.4 — Hyperparamètres de l'algorithme ADWIN⁴⁴
- Tableau 3.5 — Métriques de performance utilisées⁴⁸
- Tableau 3.6 — Stack technologique complet du projet⁵⁰
- Tableau 4.1 — Résultats de classification — comparaison des modèles⁵²
- Tableau 4.2 — Performances de détection du Concept Drift⁵⁴
- Tableau 4.3 — Métriques de performance du pipeline en temps réel⁵⁷
- Tableau 5.1 — Comparaison avec les travaux similaires de la littérature⁶³

Liste des Abréviations

Abréviation	Signification
ADWIN	ADaptive WINdowing
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
Big Data	Données massives
CDG	Concept Drift Guardrail
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DDM	Drift Detection Method
DNN	Deep Neural Network
Docker	Plateforme de conteneurisation open source
PageHinkley	Early Drift Detection Method
EMR	Elastic MapReduce (AWS)
F1	Score F1 (harmonie précision/rappel)
GDELT	Global Database of Events, Language, and Tone
GPU	Graphics Processing Unit
HuggingFace	Plateforme de modèles NLP open source
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
NoSQL	Not Only Structured Query Language
REST	Representational State Transfer
RoBERTa	Robustly Optimized BERT Pretraining Approach
ROC-AUC	Receiver Operating Characteristic — Area Under Curve
RSS	Really Simple Syndication
SLA	Service Level Agreement

TF-IDF	Term Frequency — Inverse Document Frequency
Streamlit	Framework Python de création d'applications web de data science

Chapitre 1 — Introduction Générale

1.1 Contexte et Enjeux

L'avènement d'Internet et des plateformes numériques a radicalement transformé la façon dont l'information est produite, diffusée et consommée. Si cette révolution a démocratisé l'accès au savoir et facilité la communication à l'échelle planétaire, elle a également engendré un phénomène préoccupant : la prolifération massive de la désinformation. Selon une étude publiée par Reuters Institute (2024), près de 56 % des internautes déclarent avoir été exposés à de fausses informations au cours du mois précédant l'enquête, et plus d'un tiers d'entre eux affirment avoir partagé, sciemment ou non, du contenu trompeur.

Dans ce contexte, la désinformation ne se limite plus à des erreurs anodines ou à des rumeurs locales. Elle prend la forme de campagnes coordonnées, de narratifs fabriqués de toutes pièces (« infox » ou fake news), de théories du complot virales, voire d'opérations d'influence orchestrées par des acteurs étatiques ou para-étatiques. Les conséquences observables sont multiples : manipulation des opinions publiques lors d'élections, sabotage de mesures de santé publique (comme lors de la pandémie de COVID-19), attisement de tensions interethniques ou religieuses, et déstabilisation économique par la diffusion de fausses informations boursières.

L'Afrique subsaharienne, et le Togo en particulier, ne sont pas épargnés par ce phénomène. La croissance exponentielle de l'utilisation des réseaux sociaux — en particulier WhatsApp, Facebook et Twitter/X — combinée à des niveaux d'éducation aux médias encore insuffisants, crée un terrain fertile pour la propagation rapide de fausses nouvelles. Des rumeurs pouvant déclencher des violences communautaires, fausser des processus électoraux ou compromettre des campagnes de vaccination ont été documentées dans la sous-région ces dernières années.

Face à cette menace, la Data Science et l'Intelligence Artificielle offrent des leviers technologiques puissants. Le traitement automatique du langage naturel (NLP), combiné à des architectures Big Data capables de traiter des flux de données massifs en temps réel, permet d'envisager des systèmes de veille automatisée capables de détecter et d'alerter sur la désinformation à mesure qu'elle se propage. Cependant, un défi fondamental se pose : les formes linguistiques et les thématiques de la désinformation évoluent en permanence, rendant obsolètes les modèles entraînés sur des données historiques. C'est précisément le phénomène du Concept Drift.

Le Concept Drift désigne la dérive statistique des distributions de données au fil du temps, qui se traduit par une dégradation progressive des performances des modèles

de machine learning déployés en production. Dans le contexte de la détection de désinformation, ce phénomène se manifeste concrètement lorsqu'une nouvelle rumeur émerge avec un vocabulaire, des structures grammaticales ou des thématiques inédites que le modèle n'a jamais vus lors de son entraînement. Détecter ce moment précis où le modèle commence à « dériver » constitue un enjeu scientifique et technique majeur.

1.2 Problématique

La détection automatique de la désinformation en temps réel pose plusieurs défis interdépendants qui constituent la problématique centrale de ce mémoire :

- Défi du passage à l'échelle : Les flux de données provenant de sources multiples (RSS, APIs de réseaux sociaux, web scraping) atteignent des volumes de plusieurs milliers d'articles par heure. Les architectures traditionnelles de traitement batch ne permettent pas une détection suffisamment rapide pour être utile en pratique.
- Défi de l'évolution temporelle : La désinformation évolue constamment. Un modèle de classification entraîné sur des fake news d'il y a six mois peut ne pas reconnaître les formes contemporaines d'une rumeur virale émergente, phénomène caractéristique du Concept Drift.
- Défi de la détection précoce : L'enjeu n'est pas seulement de détecter la désinformation après diffusion massive, mais d'identifier statistiquement le moment précis où une rumeur commence à diverger de la réalité et à se viraliser, permettant une intervention préventive.
- Défi des ressources : Les solutions industrielles existantes (comme ClaimBuster, Google Fact Check API, ou DARPA SemaFor) reposent sur des infrastructures coûteuses et propriétaires, inaccessibles aux organisations des pays en développement.

Ces constats nous amènent à formuler la question de recherche principale suivante :

Question de Recherche Principale

Comment concevoir un pipeline Big Data open source, économiquement accessible, capable de monitorer la désinformation en temps réel sur des flux de données hétérogènes, en intégrant un mécanisme de détection automatique du Concept Drift pour identifier statistiquement le moment où une rumeur commence à se viraliser et à diverger de la réalité ?

De cette question principale découlent trois sous-questions de recherche :

- SQ1 : Dans quelle mesure les algorithmes de détection du Concept Drift (ADWIN, PageHinkley, DDM) permettent-ils de détecter statistiquement l'émergence d'une nouvelle forme de désinformation dans un flux de données en temps réel ?
- SQ2 : Quelles sont les performances comparatives d'un modèle DistilBERT fine-tuné versus d'autres approches NLP (TF-IDF + ML classique, LSTM) pour la classification de fake news dans un contexte de streaming ?
- SQ3 : Quel gain de performance apporte le réentraînement adaptatif déclenché par la détection du Concept Drift par rapport à un modèle statique dans un environnement de production simulé ?

1.3 Objectifs et Résultats Attendus

1.3.1 Objectifs

L'objectif général de ce mémoire est de concevoir, développer et valider expérimentalement un pipeline Big Data complet et open source pour le monitoring dynamique en temps réel de la désinformation, intégrant un apprentissage en ligne continu et la détection adaptative du Concept Drift. Cet objectif général se décline en six objectifs spécifiques (OS), chacun subdivisé en sous-objectifs opérationnels précis, mesurables et directement vérifiables lors de la soutenance.

Note d'ordre chronologique d'implémentation : Les objectifs spécifiques ci-dessous sont présentés par domaine fonctionnel. L'ordre chronologique réel d'implémentation suit la séquence : (1) Infrastructure Docker + Kafka [OS6.1 → OS1.1-1.6], (2) Collecte et prétraitement des données [OS2.1-2.2], (3) Entraînement et export ONNX du modèle NLP [OS2.3-2.5], (4) Déploiement du job Spark Streaming [OS1.4], (5) Intégration du tri-détecteur de Concept Drift [OS3.1-3.6], (6) Apprentissage en ligne continu [OS4.1-4.6], (7) Monitoring Grafana + API FastAPI [OS5.1-5.3], (8) Tests et validation end-to-end [OS6.2-6.6].

OS1 — Concevoir et déployer une architecture Big Data robuste en temps réel pour l'ingestion et le traitement de flux multi-sources (RSS, GDELT, API Twitter).

- OS 1.1 : Déployer un cluster Apache Kafka 3.7 avec ZooKeeper (Confluent cp-kafka:7.6.0 / Apache Kafka 3.7, déploiement Docker Compose 11 services) via Docker Compose, avec configuration du broker en mode combiné broker+controller (KAFKA_PROCESS_ROLES=broker,controller), génération d'un Cluster UUID unique via kafka-storage.sh format, et validation du démarrage sain via kafka-topics.sh --list. Justifier le choix de KRaft par rapport à ZooKeeper en termes de simplicité opérationnelle et de scalabilité (support de millions de partitions vs ~200K avec ZooKeeper).
- OS 1.2 : Créer et configurer 3 topics Kafka avec les propriétés précises suivantes : (a) raw-news-stream — 6 partitions (1 par consumer thread Spark), rétention retention.ms=86400000 (24h), clé de partitionnement = hash(source_domain) pour

co-localiser les articles d'une même source ; (b) classified-news — 6 partitions, rétention 72h, compression.type=gzip ; (c) drift-alerts — 1 partition (ordre garanti), rétention 30 jours. Configurer min.insync.replicas=1 (environnement mono-broker Docker) et acks=all côté producer pour fiabilité maximale. Valider la topologie via Kafdrop (<http://localhost:9000>) et documenter les captures d'écran.

- OS 1.3 : Développer un Kafka Producer asynchrone en Python 3.12 (confluent-kafka 2.5.0) intégrant : (1) scraping en boucle infinie de 12 sources RSS multilingues fiables (feedparser 6.0 + httpx 0.27 async) toutes les 60 secondes avec timeout de 10s par source ; (2) requêtes à l'API GDELT GKG v2 (query=theme:FAKE_NEWS,DISINFORMATION, mode=artist, timespan=15min, format=json) toutes les 15 minutes ; (3) déduplication en mémoire par hash MD5(url+title) avec cache LRU de 100 000 IDs pour éviter les messages redondants dans le flux. Configurer le producer avec linger.ms=500 et batch.size=131072 pour optimiser le débit. Mesurer et documenter le taux de déduplication effectif sur 1 heure de collecte réelle.
- OS 1.4 : Configurer Apache Spark 3.5 Structured Streaming avec les paramètres clés suivants : trigger(processingTime='2 seconds') pour le micro-batch ; option maxOffsetsPerTrigger=1000 pour contrôler le backpressure et éviter que Spark ne prenne trop d'avance sur Kafka en cas de pic ; 2 workers Spark (bitnami/spark:3.5, 4 Go RAM, 2 cœurs chacun) ; connecteur spark-sql-kafka-0-10_2.12:3.5.1 et kafka-clients-3.5.1.jar chargés au démarrage du container. Valider la connexion Spark → Kafka en exécutant un job de lecture minimal et en vérifiant dans les logs Spark que des messages sont reçus.
- OS 1.5 : Activer et tester rigoureusement le checkpointing Spark (option checkpointLocation=/tmp/spark-checkpoints, monté dans un volume Docker persistant spark_checkpoints). Protocole de test de reprise : (a) laisser tourner le pipeline 5 minutes, (b) noter l'offset courant de chaque partition via Kafdrop, (c) arrêter brutalement le container spark-app avec docker kill, (d) le redémarrer avec docker start, (e) vérifier dans les logs que le job reprend exactement aux offsets enregistrés sans retraitement ni perte. Documenter le résultat du test avec captures de logs.
- OS 1.6 : Mettre en place le monitoring du consumer lag Kafka : configurer dans Grafana une alerte sur la métrique avgOffsetsBehindLatest > 5 000 offsets (signalant que Spark ne consomme pas assez vite). Simuler un pic de charge artificiel en injectant 1 000 messages en 30 secondes via inject_drift_simulation.py et observer : (a) la montée du lag dans Kafdrop, (b) la récupération progressive de Spark, (c) le déclenchement ou non de l'alerte. Documenter le comportement observé.
- OS 1.7 : Instrumenter la latence end-to-end du pipeline en ajoutant un champ ts_published (timestamp UTC au moment de la collecte RSS) et ts_processed

(timestamp UTC après classification Spark). Sur un échantillon de 100 articles instrumentés, calculer et documenter la distribution de la latence (médiane, P95, P99, max) avec objectif de médiane $\leq 2,5$ secondes. Produire un histogramme de distribution inclus dans le Chapitre 4 du mémoire.

OS2 — Constituer un corpus massif de 420 000+ exemples (après fusion et déduplication des 4 datasets) (après fusion et déduplication des 4 datasets disponibles) et développer un modèle NLP Continual-DistilBERT haute performance pour la classification dynamique de fake news.

- OS 2.1 : Télécharger et vérifier l'intégrité des 4 datasets publics via le script automatisé `download_datasets.sh` : (a) ISOT — 44 898 articles Reuters + fake (Université de Victoria, zip direct) ; (c) WELFake — 72 134 articles (Zenodo, DOI 10.5281/zenodo.4561253, vérification du hash SHA256 publié) ; (d) FakeNewsNet — 23 196 articles (HuggingFace Hub, GonzaloA/fake_news, chargement via `datasets.load_dataset`) ; (e) LIAR — 12 836 déclarations politiques (UCSB, fichiers TSV). Produire un rapport de vérification listant le nombre de lignes effectif vs attendu pour chaque source.
- OS 2.2 : Implémenter le script `preprocess_data.py` exécutant les 8 étapes de prétraitement : (1) Normalisation Unicode NFC et suppression des caractères de contrôle ; (2) Nettoyage web/social : URLs $\rightarrow$ [URL], @mentions $\rightarrow$ [USER], décodage entités HTML ; (3) Harmonisation multilingue par détection automatique de la langue (`langdetect`) ; (4) Construction du champ d'entrée unifié titre + [SEP] + corps[:200 tokens] avec `max_length` adaptatif (128 tokens pour Fakeddit/LIAR, 256 pour ISOT/WELFake/FakeNewsNet) ; (5) Binarisation des labels LIAR (true/mostly-true/half-true $\rightarrow$ 0, sinon $\rightarrow$ 1) ; (6) Déduplication par hash MD5 sur les titres avec logging du nombre de doublons supprimés ; (7) Split temporel stratifié 80/10/10 (par source+label) ; (8) Génération du rapport de qualité : distribution labels, longueurs des textes, langues détectées. Valider que le corpus final contient $\geq 1\,150\,000$ exemples uniques.
- OS 2.3 : Pré-entraîner le backbone `distilbert-base-multilingual-cased` (134M paramètres, 104 langues) sur le corpus de train via AdamW (`weight_decay=0.01`), Layer-wise Learning Rate Decay (facteur 0.9 par couche depuis la dernière couche du Transformer vers les embeddings), gradient clipping `max_norm=1.0`, et pondération des classes (`weight_fake = n_real/n_total`, `weight_real = n_fake/n_total`). Entraîner sur 10 epochs avec early stopping `patience=2` sur le F1-macro de validation. Exécuter sur Google Colab T4 GPU (16 Go VRAM, 6h

estimées) ou A100 (Colab Pro+, 2.5h). Sauvegarder les checkpoints toutes les 2 epochs dans `models/checkpoints/`.

- OS 2.4 : Adapter le classification head du modèle pour l'apprentissage continu en ligne : remplacer la tête linéaire standard `Linear(768→2)` par la séquence `Linear(768→512) → LayerNorm → GELU → Dropout(0.2) → Linear(512→2)`. Justifier l'ajout de `LayerNorm` pour la stabilisation des gradients lors des mises à jour online à petits batches, et du `Dropout(0.2)` pour le maintien de la régularisation. Mesurer l'impact de ce head étendu vs le head standard sur le F1-score de validation (A/B test documenté dans le Tableau 3.3).
- OS 2.5 : Implémenter le Reservoir Buffer (algorithme de Vitter, 1985) avec capacité de 5 000 exemples, couvrant approximativement 4 heures de flux à 1 200 articles/heure. Configurer le ratio replay dynamique : 20 % d'exemples de replay quand `composite_score < 0.50` (régime stable, `lr_base=1e-5`) et 50 % quand `composite_score ≥ 0.50` (régime de dérive, `lr_drift=5e-5`). Valider l'absence d'oubli catastrophique par test dédié : (a) pré-entraîner sur 10 000 exemples domaine politique (Fakeddit-politics), (b) faire 5 000 mises à jour en ligne sur domaine santé (Fakeddit-health), (c) mesurer le F1-score sur un holdout politique — la chute doit être ≤ 3 points F1 pour valider l'efficacité du replay.
- OS 2.6 : Exporter le modèle PyTorch vers ONNX via HuggingFace Optimum (`ORTModelForSequenceClassification.from_pretrained(path, export=True)`), puis appliquer la quantification dynamique INT8 (`AutoQuantizationConfig.arm64, is_static=False, per_channel=False`). Benchmarker la latence sur 1 000 inférences séquentielles (article par article, sans batching) sur CPU et produire la distribution (médiane, P95, P99, max). Mesurer l'écart de F1-score entre FP32 et INT8 sur le jeu de test — régression acceptable $\leq 0,3$ point F1. Sauvegarder `model_quantized.onnx` (~65 Mo) dans `models/onnx/` et documenter les deux métriques (latence + F1) dans le Tableau 3.4.
- OS 2.7 : Intégrer le Continual-DistilBERT dans le job Spark via une architecture hybride : inférence ONNX distribuée (pandas UDF appliquée sur les workers Spark, session `InferenceSession` chargée au premier appel et mise en cache) + mise à jour online PyTorch centralisée (dans le driver Spark, appliquée via `foreachBatch` après collecte). Mesurer séparément : (a) le temps d'inférence moyen par article via la pandas UDF (objectif ≤ 7 ms sur CPU) ; (b) le temps

total de mise à jour online par micro-batch (objectif ≤ 500 ms pour 64 articles + replay). Valider que le throughput global atteint ≥ 200 articles/seconde.

OS3 — Implémenter, configurer et comparer rigoureusement un tri-détecteur de Concept Drift (ADWIN, KSWIN, PageHinkley) intégré nativement au flux Spark.

- OS 3.1 : Implémenter ADWIN (ADaptive WINdowing, Bifet & Gavalda 2007) via River 0.21 (drift.ADWIN) avec $\text{delta}=0.002$, $\text{clock}=32$, $\text{min_window_length}=5$. Alimenter ADWIN avec la série temporelle du score $P(\text{fake}|\text{texte})$ en sortie du modèle ONNX. Calibrer le paramètre delta par recherche par grille sur $\{0.001, 0.002, 0.005, 0.01\}$: pour chaque valeur, mesurer (a) le FPR sur 2 000 articles sans dérive et (b) le délai de détection moyen sur le Scénario A (dérive abrupte). Retenir $\text{delta}=0.002$ comme meilleur compromis FPR/délai et documenter la justification quantitative dans un tableau de calibration.
- OS 3.2 : Implémenter KSWIN (Kolmogorov-Smirnov WINdowing, Raab et al. 2020) via River 0.21 (drift.KSWIN) avec $\text{window_size}=100$, $\text{stat_size}=30$ (30 % de la fenêtre), $\text{alpha}=0.005$. Alimenter KSWIN avec la même série $P(\text{fake}|\text{texte})$ pour détecter les changements de distribution statistique — y compris les dérives virtuelles où la performance du modèle ne se dégrade pas immédiatement mais où la distribution des données d'entrée change. Illustrer la complémentarité ADWIN (moyenne) vs KSWIN (distribution) sur le Scénario D (dérive incrémentale) via un graphique dual dans le Chapitre 4.
- OS 3.3 : Implémenter PageHinkley via River 0.21.2 (drift.PageHinkley) avec $\text{min_instances}=30$, $\text{delta}=0.005$, $\text{threshold}=50.0$. Alimenter PageHinkley avec la série temporelle du score de confiance $P(\text{fake})$ pour détecter les dérives graduelles dans la distribution des prédictions en confrontant les prédictions du modèle aux pseudo-labels automatiques de la GDELT Fact Check API pour le sous-ensemble d'articles couverts (estimation : 15-20 % des articles scrappés). Documenter le taux de couverture réel des pseudo-labels sur 1 heure de flux et discuter son impact sur la fiabilité d'PageHinkley.
- OS 3.4 : Développer la classe DynamicDriftMonitor centralisant les 3 détecteurs avec le score composite pondéré $S = \text{ADWIN} \times 0.45 + \text{KSWIN} \times 0.35 + \text{PageHinkley} \times 0.20$. Calibrer les poids par validation exhaustive sur 12 combinaisons (grille $\{0.45, 0.50\} \times \{0.30, 0.35\} \times \{0.15, 0.20\}$) en maximisant la somme $\text{TPR} - 2 \times \text{FPR}$ sur les 4 scénarios combinés. Documenter la combinaison retenue avec son score et les 3 alternatives les plus proches. Configurer deux seuils : $S > 0.50 \rightarrow$ mode Dérive Active (modulation lr + enrichissement buffer), $S > 0.80 \rightarrow$ Dérive Confirmée (re-synchronisation ONNX).

- OS 3.5 : Construire les 4 scénarios de dérive simulés à partir des jeux de données Fakeddit + WELFake pour leur diversité lexicale et thématique : (A) Abrupt — injection instantanée à $t=5\,000$ de 500 articles fake simulant une crise sanitaire inédite (vocabulaire médical jamais vu lors du pré-entraînement) ; (B) Graduel — proportion de fake croît de 0 % à 80 % linéairement sur 3 000 messages, simulant une campagne politique montant en puissance ; (C) Récurrent — alternance thématique politique ↔ santé toutes les 2 500 messages sur 4 cycles, testant la mémoire du modèle sur les domaines passés ; (D) Incrémental — glissement lexical progressif et monotone sur 5 000 messages. Pour chaque scénario $\times$ algorithme, mesurer et enregistrer : délai de détection moyen (msgs), TPR, FPR.
- OS 3.6 : Configurer l'émission automatique des alertes de dérive vers le topic Kafka drift-alerts dès que $S > 0.50$. Définir le schéma JSON standardisé du payload : {timestamp:ISO8601, composite_score:float, signals:{ADWIN:bool, KSWIN:bool, PageHinkley:bool}, drift_confirmed:bool, recommended_lr:float, messages_total:int, confidence_mean_last100:float}. Valider : (a) persistance dans MongoDB collection drift_events ; (b) indexation dans Elasticsearch index drift-events ; (c) affichage en temps réel dans Grafana panneau P4 (Table). Tester le mécanisme de cooldown (1 000 messages) en vérifiant qu'une seule alerte est émise pour une dérive continue, pas une rafale d'alertes répétitives.

OS4 — Implémenter et valider le paradigme d'apprentissage en ligne continu avec modulation dynamique du taux d'apprentissage pilotée par le Concept Drift.

- OS 4.1 : Implémenter la boucle d'apprentissage online dans la fonction process_batch() du job Spark : à chaque micro-batch de 2 secondes, après la phase d'inférence ONNX, effectuer une passe de gradient unique (single epoch, pas de boucle) sur les n articles du batch courant + $\max(1, n/4)$ exemples tirés aléatoirement du Reservoir Buffer (ratio replay 20 % en régime stable). Conserver l'état complet de l'optimiseur AdamW (tenseurs m_t et v_t des moments) entre les micro-batches pour assurer la continuité de l'optimisation — ne pas réinitialiser l'optimiseur entre batches. Logger la loss online de chaque micro-batch dans MongoDB pour visualisation dans Grafana.
- OS 4.2 : Implémenter la modulation dynamique du learning rate pilotée par le score composite S : (a) $lr = lr_base = 1e-5$ quand $S < 0.50$ (flux stable) ; (b) lr passe instantanément à $lr_drift = 5e-5$ (facteur $\times 5$) dès $S \geq 0.50$, accélérant l'adaptation aux nouvelles formes de désinformation ; (c) retour progressif à lr_base via un scheduler linéaire sur 1 000 messages de cooldown après que S

repassse sous 0.50. Mesurer la vitesse de récupération du F1-score avec lr dynamique vs lr fixe à 1e-5 sur le Scénario B : enregistrer le F1 prequential à T0+500, T0+1000 et T0+2000 messages pour les deux configurations.

- OS 4.3 : Implémenter le Mixup online adaptatif (Zhang et al. 2018, ICLR) : pour chaque micro-batch, construire $\text{ceil}(n/4)$ exemples synthétiques par interpolation convexe $\lambda \cdot x_i + (1-\lambda) \cdot x_j$ entre paires d'articles de classes opposées ($\text{is_fake}=0 \leftrightarrow \text{is_fake}=1$), avec $\lambda \sim \text{Beta}(0.2, 0.2)$. Inclure ces exemples dans le batch d'entraînement online. Évaluer l'impact du Mixup par comparaison A/B sur les 4 scénarios : mesurer le F1 prequential à T0+1000 messages post-dérive avec et sans Mixup, et documenter le gain éventuel.
- OS 4.4 : Évaluer le Continual-DistilBERT (dynamique) vs le DistilBERT statique (figé post-pré-entraînement, aucune mise à jour) via le protocole prequential (test-then-train) sur le Scénario B. Mesurer le F1-score prequential (fenêtre glissante $W=1\ 000$) à exactement 6 instants : T0 (baseline pré-dérive), T0+200, T0+500, T0+1 000, T0+2 000 et T0+3 000 messages post-dérive. Produire le Tableau 4.2 du mémoire avec les deux colonnes (Statique vs Dynamique) et la colonne $\Delta\text{-F1}$, et tracer les deux courbes d'évolution dans la Figure 4.3.
- OS 4.5 : Implémenter la re-synchronisation périodique du modèle ONNX depuis les poids PyTorch mis à jour en ligne. Déclencher automatiquement une re-quantification INT8 (ORTQuantizer) toutes les 12 heures ou dès que $\text{composite_score} > 0.80$ (Dérive Confirmée). Pendant la re-synchronisation (durée cible < 5 minutes), maintenir le streaming opérationnel en servant les inférences via le modèle ONNX de la version précédente (mécanisme de blue-green hot-swap : ancien modèle en service, nouveau en construction, basculement atomique à la fin). Mesurer et documenter le temps de re-synchronisation et valider l'absence d'interruption du flux Kafka (lag mesuré pendant l'opération).
- OS 4.6 : Démontrer et quantifier expérimentalement le paradigme 'dérive = opportunité d'apprentissage' : vérifier que le F1-score du Continual-DistilBERT à T0+3 000 messages post-dérive graduelle est strictement supérieur au F1-score baseline pré-dérive (T0). Ce phénomène, résultant de l'adaptation continue du modèle à la nouvelle distribution des données, constitue la contribution scientifique principale du mémoire et sera mis en valeur dans la conclusion comme un résultat contre-intuitif et novateur. Proposer une interprétation théorique basée sur l'effet de domain adaptation continu.

OS5 — Concevoir, déployer et valider un tableau de bord de monitoring en temps réel sous Grafana 10 avec système d'alertes configurable.

- OS 5.1 : Configurer les 3 datasources Grafana 10 via provisioning automatique YAML (config/grafana/provisioning/datasources/datasources.yaml) pour garantir un déploiement 100 % reproductible : (a) Elasticsearch 8.14 — URL `http://elasticsearch:9200`, index `articles`, timeField `processed_at`, esVersion `8.x`, logMessageField `title`, logLevelField `is_fake` ; (b) Elasticsearch-Drift — index `drift-events`, timeField `timestamp` ; (c) DisinformationAPI — plugin `marcusolsson-json-datasource`, URL `http://api:8000`, pour les métriques en temps réel du modèle (F1 prequential, learning rate, loss online). Valider chaque datasource via le bouton 'Test connection' dans Grafana et documenter le statut.
- OS 5.2 : Développer les 7 panneaux du dashboard principal (refresh 10s, fenêtre par défaut 6h, provisioning auto depuis config/grafana/dashboards/) : (P1) Gauge Taux de Fakes (%) avec seuils vert/orange/rouge à 35 %/50 % ; (P2) Time Series Flux d'articles/heure groupé par source_category (reliable vs suspicious) avec annotations des dérives ; (P3) Time Series Score Composite Drift avec 3 zones colorées (vert < 0.5, orange 0.5-0.8, rouge > 0.8) et marqueurs des événements de drift ; (P4) Table Historique des 20 dernières alertes (timestamp, composite_score, détecteurs déclenchés, drift_confirmed) ; (P5) Stat Learning Rate Courant avec coloration contexte (bleu=stable, orange=actif) ; (P6) Pie Chart Répartition des articles fake par domaine source (top 10, filtre `is_fake=1`) ; (P7) Time Series F1-score Prequential glissant (fenêtre W=1 000 derniers exemples).
- OS 5.3 : Configurer 3 règles d'alerte Grafana avec conditions de déclenchement précises : (A1) Taux de fakes moyen sur 5 min > 50 % → alerte critique 'Vague de désinformation détectée' (notification canal Grafana + message dans MongoDB) ; (A2) Score composite drift > 0.80 pendant 2 mesures consécutives (20 s) → alerte critique 'Dérive Confirmée — Adaptation accélérée activée' ; (A3) F1-score prequential moyen < 0.85 sur 10 min → alerte avertissement 'Dégradation du modèle — Vérification requise'. Tester chaque règle en déclenchant volontairement la condition et documenter la notification reçue avec capture d'écran.
- OS 5.4 : Valider le provisioning automatique complet après une réinstallation totale : exécuter `docker compose down -v` (suppression de tous les volumes) puis `docker compose up -d`, et vérifier que le dashboard avec ses 7 panneaux est automatiquement rechargé sans aucune intervention manuelle. Mesurer le temps de chargement initial du dashboard depuis le démarrage de Grafana. Tester la performance des requêtes sur une fenêtre de 6h contenant $\geq 50\,000$ documents (objectif : toutes les requêtes ≤ 2 secondes) et documenter le temps de réponse de chaque panneau.

OS6 — Valider le pipeline complet en environnement Docker conteneurisé, exécuter le scénario de soutenance end-to-end et contribuer un dépôt open source documenté.

- OS 6.1 : Dockeriser les 4 services applicatifs maison (rss-producer, spark-app, api, spark-worker) avec des Dockerfiles optimisés : image python:3.12-slim pour producer et api, image bitnami/spark:3.5 pour le job streaming (avec packages JAR Kafka téléchargés au build via curl). Orchestrer les 11 services (Zookeeper, Kafka, Kafdrop, Spark Master, Spark Worker×2, MongoDB, Elasticsearch, Grafana, FastAPI) via docker-compose.yml avec healthchecks sur chaque service critique (Kafka, MongoDB, Elasticsearch) et chaînes depends_on appropriées pour garantir le bon ordre de démarrage.
- OS 6.2 : Valider le déploiement reproductible depuis une machine vierge (satisfaisant les prérequis RAM/CPU/Docker) en suivant uniquement les 5 commandes du README : git clone → bash scripts/download_datasets.sh → python scripts/train_model.py → python scripts/export_onnx.py → docker compose up -d. Mesurer et documenter le temps total de déploiement opérationnel (objectif : < 2 heures hors pré-entraînement). Réaliser ce test sur au minimum deux environnements distincts (Ubuntu 22.04 + macOS 14 ou Windows 11/WSL2) et documenter les éventuelles différences.
- OS 6.3 : Exécuter le scénario de validation end-to-end de la soutenance en 6 actes vérifiables : lancer inject_drift_simulation.py (200 articles, rate=0.1 s/msg) et valider simultanément — (a) arrivée des messages dans Kafka (Kafdrop : offsets croissants sur raw-news-stream) ; (b) traitement Spark (logs 'Batch N: X articles', 'Online loss: X.XXX', 'lr: X.Xe-0X') ; (c) classification NLP correcte (API /api/v1/articles/recent → is_fake=1 pour les articles injectés, confidence > 0.85) ; (d) montée du score composite drift > 0.50 dans Grafana P3 en < 8 minutes ; (e) modulation lr 1e-5 → 5e-5 visible dans Grafana P5 ; (f) alerte émise dans le topic drift-alerts (kafka-console-consumer) et dans MongoDB. Capturer chaque étape pour la présentation devant le jury.
- OS 6.4 : Mesurer et documenter les métriques de performance finales sur une session de test de 2 heures de fonctionnement continu en conditions réelles (flux RSS actif) : (a) latence end-to-end médiane et P95 (objectif : médiane ≤ 2,5 s) ; (b) throughput maximal observé (objectif : ≥ 200 articles/s en simulation) ; (c) utilisation mémoire par container (docker stats, max observé sur 2h) ; (d) consumer lag Kafka max observé sur 2h (objectif : < 5 000 offsets) ;

(e) taux de disponibilité des 9 services (objectif : 100 % sur 2h en conditions normales). Compiler ces métriques dans le Tableau 4.3 du Chapitre 4.

- OS 6.5 : Publier le dépôt GitHub public sous licence MIT avec les livrables documentés suivants : (a) README.md avec badges (Python 3.12, Kafka 3.7, Spark 3.5, Docker, License MIT), description du projet, schéma d'architecture, et guide d'installation en 5 étapes ; (b) scripts/ complets et commentés ; (c) requirements.txt par service ; (d) docker-compose.yml avec tous les healthchecks et commentaires inline ; (e) config/ avec tous les fichiers de provisioning Grafana, MongoDB init.js, mapping Elasticsearch ; (f) notebooks/ incluant 01_data_exploration.ipynb, 02_model_training.ipynb, 03_evaluation.ipynb. Citer le lien du dépôt comme contribution open source dans la section Conclusion du mémoire.

1.3.2 Résultats Attendus

À l'issue de ce travail, les résultats suivants sont attendus, organisés par objectif spécifique. Chaque résultat constitue un livrable précis, mesurable et directement vérifiable lors de la soutenance. Les valeurs cibles sont issues de l'état de l'art 2024-2025 et justifiées par les benchmarks de la littérature :

RA1 — Architecture Big Data opérationnelle et validée :

- RA 1.1 : Cluster Kafka 3.7 (mode KRaft, sans ZooKeeper) opérationnel avec les 3 topics configurés selon les spécifications exactes (6/6/1 partitions, rétentions 24h/72h/30j, compression gzip sur classified-news). Interface Kafdrop accessible sur <http://localhost:9000> avec visualisation en temps réel des offsets, des consumer groups et du lag. Débit de collecte mesuré ≥ 60 messages/minute en régime nominal.
- RA 1.2 : Kafka Producer opérationnel et résilient : 12 sources RSS scrapées toutes les 60s + GDELT GKG v2 interrogée toutes les 15 min. Taux de déduplication documenté sur 1 heure réelle (attendu : 30-50 % de doublons supprimés entre les sources). Fréquence de collecte effective ≥ 95 % des intervalles de 60s respectés mesurée et documentée.
- RA 1.3 : Job Spark Structured Streaming opérationnel en micro-batches de 2 secondes, 2 workers actifs visibles dans Spark Master UI (<http://localhost:8080>). Checkpointing validé : reprise exacte documentée après arrêt brutal du container. Throughput mesuré ≥ 200 articles/seconde

dans les conditions simulées de charge (inject_drift_simulation.py, 1 000 msgs/30s).

- RA 1.4 : Latence end-to-end documentée avec distribution complète : médiane $\leq 2,5$ s, P95 ≤ 4 s sur 100 articles instrumentés. Consumer lag maintenu ≤ 5 000 offsets en régime nominal avec alerte Grafana testée et validée. Histogramme de distribution de latence inclus dans la Figure 4.1 du mémoire.

RA2 — Modèle NLP haute performance sur corpus massif :

- RA 2.1 : Corpus fusionné de $\geq 1\,150\,000$ exemples uniques constitué et validé. Rapport de qualité documenté : distribution labels ($\approx 58,7$ % fake / $\approx 41,3$ % réel), nombre exact de doublons supprimés, répartition par source (Fakeddit dominant avec >91 %), distribution des longueurs de textes et des langues détectées. Fichiers CSV train/val/test disponibles avec leurs checksums MD5.
- RA 2.2 : Continual-DistilBERT pré-entraîné atteignant F1-score Macro ≥ 91 % sur le jeu de test (2023-2024, 116 917 exemples). Tableau comparatif complet de 6 baselines (TF-IDF+LR, TF-IDF+RF, BiLSTM+GloVe, BERT-base-uncased, RoBERTa-base, DistilBERT statique) avec colonnes F1 Macro, Précision, Rappel, ROC-AUC et Latence. Matrice de confusion à 4 cellules (TP, FP, FN, TN) et courbe d'apprentissage (loss + F1 par epoch, 10 epochs) incluses dans les Figures 4.2 et 4.1 respectivement.
- RA 2.3 : Modèle ONNX INT8 exporté (taille ~ 65 Mo). Benchmark de latence sur 1 000 inférences : médiane ≤ 6 ms/article, distribution P95 et max documentés. Régression F1 FP32 vs INT8 $\leq 0,3$ point (validant l'absence de perte de qualité significative lors de la quantification). Ces deux métriques constituant les critères de validation de l'OS 2.6 sont incluses dans le Tableau 3.4.
- RA 2.4 : Test anti-catastrophic forgetting documenté et validé : F1 sur le domaine A après 5 000 mises à jour sur le domaine B chute de ≤ 3 points, confirmant l'efficacité du Reservoir Buffer. Surcoût de la pandas UDF mesuré < 50 ms/micro-batch. Temps de re-synchronisation ONNX post-dérive < 5 minutes avec streaming maintenu continu (hot-swap validé et documenté).

RA3 — Tri-détecteur de Concept Drift robuste et calibré :

- RA 3.1 : Tableau comparatif exhaustif de 12 mesures (3 algorithmes $\times$ 4 scénarios) avec délai de détection moyen, TPR et FPR pour chaque combinaison. Mise en évidence claire de la complémentarité : ADWIN le plus équilibré globalement, KSWIN le plus performant sur le Scénario D (dérive incrémentale), PageHinkley le plus rapide sur le Scénario A (dérive abrupte) mais avec FPR le plus élevé.
- RA 3.2 : Score composite ($\text{ADWIN} \times 0.45 + \text{KSWIN} \times 0.35 + \text{PageHinkley} \times 0.20$) atteignant $\text{TPR} \geq 0.97$ et $\text{FPR} \leq 0.03$ sur les Scénarios A et B, avec délai ≤ 63 messages ($\approx 7,6$ minutes à 500 articles/heure) sur le Scénario B — validant l'intervention préventive 12-22 minutes avant la masse critique virale (estimée à 20-30 min selon Vosoughi et al., Science 2018).
- RA 3.3 : Calibration des poids documentée : grille de 12 combinaisons testées avec leurs scores $\text{TPR} - 2 \times \text{FPR}$, justification de la combinaison retenue (0.45/0.35/0.20) et des seuils (0.50/0.80). Mécanisme de cooldown validé : une seule alerte émise pour une dérive continue de 10 minutes (pas de rafale).
- RA 3.4 : Infrastructure d'alertes complète et testée : alertes persistées dans MongoDB (drift_events), indexées dans Elasticsearch (drift-events), affichées en temps réel dans Grafana P4, et consommables depuis le topic Kafka drift-alerts — les 4 canaux validés et documentés avec captures d'écran.

RA4 — Online Learning dynamique — contribution scientifique principale :

- RA 4.1 : Tableau 4.2 du mémoire complété : F1-score prequential de DistilBERT statique vs Continual-DistilBERT dynamique aux 6 instants de mesure sur le Scénario B, avec colonne $\Delta\text{-F1}$. Gain $\geq +20$ points F1 à T0+3 000 messages documenté. Courbes d'évolution (Figure 4.3) montrant clairement la divergence entre les deux modèles à partir de T0+500 messages.
- RA 4.2 : Démonstration quantifiée et reproductible du paradigme 'dérive = opportunité' : F1 dynamique post-adaptation (T0+3 000 msgs) $>$ F1 baseline pré-dérive (T0) de +0.9 point ou plus. Ce résultat, contre-intuitif vis-à-vis de la littérature classique sur le Concept Drift qui traite uniquement les dérives comme des menaces, constitue la contribution scientifique principale et sera discuté dans la Section 5.2 (Comparaison avec la littérature).
- RA 4.3 : Modulation dynamique du lr tracée et visible lors de la soutenance dans Grafana P5 : transition $1e-5 \rightarrow 5e-5$ déclenchée en direct lors de l'injection de drift, retour progressif à $1e-5$ après cooldown. Comparaison lr fixe vs lr dynamique sur le Scénario B documentée (gain de vitesse de récupération mesuré en nombre de messages supplémentaires nécessaires).

RA5 — Dashboard Grafana temps réel professionnel :

- RA 5.1 : Dashboard Grafana 10 opérationnel avec 7 panneaux actifs, données en direct (refresh 10s), fenêtre par défaut 6h. Les 3 règles d'alertes configurées, testées et documentées avec captures d'écran des notifications. Provisioning automatique validé après docker compose down -v && docker compose up -d : dashboard rechargé en < 2 minutes sans intervention manuelle. Dashboard Streamlit opérationnel sur <http://localhost:8501> avec 5 pages multi-vues (Vue d'ensemble, Explorer les articles, Monitoring Drift, Analyse des sources, Configuration). Auto-refresh toutes les 30 secondes.
- RA 5.2 : Performance validée : temps de réponse de tous les panneaux Grafana ≤ 2 secondes mesuré sur une fenêtre de 6h avec $\geq 50\,000$ documents indexés. Le panneau P3 (Score Composite Drift) affiche clairement la montée du score lors de la démonstration devant le jury, constituant la visualisation la plus impactante de la soutenance.

RA6 — Pipeline validé, métriques documentées et contribution open source :

- RA 6.1 : Pipeline déployable en < 2 heures (hors pré-entraînement) depuis une machine vierge, validé sur deux environnements distincts. Checklist de vérification des 11 services documentée avec captures d'écran pour la soutenance.
- RA 6.2 : Scénario de soutenance exécuté end-to-end avec succès : les 6 étapes validées et documentées par captures d'écran. Le script `inject_drift_simulation.py` exécutable en une seule commande (`python scripts/inject_drift_simulation.py`) devant le jury avec résultats visibles dans Grafana en < 8 minutes.
- RA 6.3 : Tableau 4.3 du mémoire complété avec métriques de performance finales : latence médiane, P95, throughput, lag max, uptime sur 2h. Ces métriques constituant les critères quantitatifs de validation de l'objectif général.
- RA 6.4 : Dépôt GitHub public publié sous licence MIT avec README complet (badges, architecture, guide 5 étapes), scripts annotés, configuration Docker reproductible, et 3 notebooks Jupyter. URL du dépôt citée dans la Conclusion du mémoire comme contribution open source réutilisable par la communauté scientifique, en particulier pour les institutions de lutte contre la désinformation en Afrique de l'Ouest.

1.4 Méthodologie Générale

Ce projet adopte une approche méthodologique mixte, combinant une démarche quantitative expérimentale pour l'évaluation des performances et une démarche

ingénierie pour la conception et le développement du système. La recherche est structurée en quatre phases principales :

- Phase 1 — Revue de littérature et positionnement : Analyse exhaustive des travaux scientifiques sur la détection de fake news, le Concept Drift dans les flux de données, et les architectures Big Data temps réel, afin d'identifier les lacunes et de justifier les choix architecturaux.
- Phase 2 — Conception et développement : Implémentation itérative du pipeline, du scraper RSS au tableau de bord Grafana, en passant par les modules Kafka, Spark, NLP et Concept Drift Detection.
- Phase 3 — Expérimentation et validation : Évaluation quantitative des performances du système selon des métriques standardisées (F1-score, latence, throughput, délai de détection du drift), sur des datasets publics et des scénarios de simulation.
- Phase 4 — Analyse et synthèse : Interprétation des résultats, comparaison avec la littérature, identification des limites et formulation de recommandations.

Les technologies utilisées sont toutes open source et gratuites : Python 3.12+, Apache Kafka 3.7, Apache Spark 3.5, River 0.21, HuggingFace Transformers 4.40, Docker 26, Grafana 10, MongoDB 7, Elasticsearch 8.

1.5 Structure du Document

Ce mémoire est organisé en six chapitres. Le Chapitre 1 (présent) établit le contexte, la problématique, les objectifs et la méthodologie générale. Le Chapitre 2 propose une revue de littérature approfondie couvrant la détection de fake news, le Concept Drift et les architectures Big Data temps réel. Le Chapitre 3 détaille la méthodologie technique, de la présentation des données à l'architecture complète du pipeline. Le Chapitre 4 expose et analyse les résultats expérimentaux obtenus. Le Chapitre 5 propose une discussion critique des résultats, comparaison avec la littérature et recommandations. Enfin, le Chapitre 6 conclut le mémoire en synthétisant les contributions et en proposant des perspectives de recherche future. Les annexes fournissent les codes sources complets et les configurations techniques détaillées.

Chapitre 2 — Revue de la Littérature

2.1 Introduction à la Littérature

La lutte contre la désinformation à l'ère numérique se situe à l'intersection de plusieurs domaines scientifiques en pleine effervescence : le traitement automatique du langage naturel (NLP), les systèmes Big Data en temps réel, et l'apprentissage automatique adaptatif. Pour comprendre le positionnement de notre travail, il est indispensable de retracer l'évolution historique de ces domaines et de cartographier les avancées récentes qui constituent l'état de l'art actuel.

La détection automatique de la désinformation a connu une accélération spectaculaire à partir de 2016, période marquée par plusieurs campagnes de désinformation d'ampleur lors d'élections présidentielles en Europe et aux États-Unis. Les premiers systèmes reposaient sur des approches linguistiques classiques (sac de mots, TF-IDF) et des classifieurs simples (SVM, forêts aléatoires). L'avènement des réseaux de neurones profonds, et en particulier des modèles Transformer (BERT, 2018 ; RoBERTa, 2019 ; DistilBERT, 2019), a profondément transformé le domaine, permettant des performances de classification nettement supérieures grâce à la capture de contextes sémantiques complexes.

Parallèlement, le domaine du Big Data en temps réel a connu une maturation significative. L'émergence d'Apache Kafka (2011, LinkedIn) comme standard de messagerie distribuée, suivie d'Apache Spark Structured Streaming (2016) puis d'Apache Flink, a permis le traitement de flux de données à l'échelle de milliards d'événements par jour avec des latences sub-secondes. Ces architectures ont trouvé des applications dans la détection de fraude financière, la surveillance des réseaux, et plus récemment dans la veille informationnelle.

Le Concept Drift, phénomène étudié depuis les travaux fondateurs de Gama et al. (2004) avec le DDM, a connu un regain d'intérêt considérable avec la généralisation des systèmes de machine learning en production. La bibliothèque River (2020, anciennement scikit-multiflow), qui unifie les algorithmes d'apprentissage en ligne et de détection du drift, représente l'outil de référence actuel dans ce domaine.

2.2 Concepts Clés

2.2.1 La Désinformation : Définitions et Typologies

La littérature distingue plusieurs concepts voisins qu'il convient de clarifier avec précision :

Concept	Définition
Misinformation	Fausse information diffusée sans intention délibérée de tromper
Disinformation	Fausse information créée et diffusée délibérément pour induire en erreur
Fake News	Contenu journalistique fabriqué, imitant la forme d'une vraie information
Rumeur	Information non vérifiée circulant dans un réseau social
Infoux	Terme francophone équivalent à fake news, consacré par l'Académie française
Deepfake	Contenu multimédia falsifié par IA (image, vidéo, audio)

Wardle & Derakhshan (2017) proposent une taxonomie en sept catégories de contenu trompeur, allant de la satire (non intentionnellement trompeuse) à la propagande (contenu fabriqué de toutes pièces). Dans le cadre de ce mémoire, nous nous concentrons sur la détection automatique des catégories fake news (contenu journalistique falsifié) et rumeurs virales (informations non vérifiées présentant des patterns de propagation exponentiels).

2.2.2 Le Concept Drift : Théorie et Taxonomie

Le Concept Drift est formellement défini comme la modification de la distribution conditionnelle $P(y|X)$ au fil du temps, où X représente les variables d'entrée (features) et y la variable cible (label). Gama et al. (2004) formalisent ce phénomène : on parle de dérive au temps t si la distribution $P_t(X,y)$ diffère significativement de $P_{t+\Delta t}(X,y)$ pour un certain intervalle de temps Δt .

La taxonomie standard du Concept Drift distingue quatre types principaux :

- Dérive abrupte (Abrupt Drift) : Changement brutal et soudain de la distribution. Exemple : émergence soudaine d'une nouvelle terminologie de désinformation liée à un événement imprévu (catastrophe naturelle, conflit armé, pandémie).
- Dérive graduelle (Gradual Drift) : Transition lente entre deux concepts. Exemple : évolution progressive du vocabulaire d'une campagne de désinformation politique sur plusieurs semaines.
- Dérive incrémentale (Incremental Drift) : Changement continu et monotone. Exemple : glissement progressif des thèmes de désinformation au fil d'une campagne électorale prolongée.
- Dérive récurrente (Recurring Drift) : Réapparition périodique de concepts passés. Exemple : résurgence annuelle de théories conspirationnistes liées à des événements calendaires.

Hinder et al. (2024), dans une revue de référence publiée dans *Frontiers in Artificial Intelligence*, soulignent que la plupart des études se concentrent sur les flux supervisés, alors que la détection non supervisée — particulièrement pertinente pour le monitoring de la désinformation où les labels arrivent avec délai — reste un domaine insuffisamment exploré.

2.2.3 Algorithmes de Détection du Concept Drift

Les principaux algorithmes de détection du Concept Drift peuvent être classés en deux catégories : les méthodes basées sur la performance (error-based) et les méthodes basées sur la distribution des données (window-based).

Algorithme	Description et caractéristiques principales
DDM (Gama et al., 2004)	Surveille le taux d'erreur du classifieur comme variable de Bernoulli. Déclenche une alerte quand $p_i + s_i \geq p_{min} + 2 \times s_{min}$ et une dérive quand $p_i + s_i \geq p_{min} + 3 \times s_{min}$. Efficace pour les dérives abruptes, moins adapté aux dérives graduelles.
PageHinkley (Baena-García et al., 2006)	Extension de DDM améliorant la détection des dérives graduelles en surveillant la distance entre erreurs consécutives. Plus sensible mais génère plus de faux positifs.
ADWIN (Bifet & Gavalda, 2007)	Fenêtre adaptative glissante. Compare les statistiques de deux sous-fenêtres selon l'inégalité de Hoeffding. Considéré comme le gold standard pour sa robustesse et sa polyvalence.
HDDM (Frías-Blanco et al., 2015)	Extension de DDM utilisant l'inégalité de Hoeffding. Variantes HDDM_A (test A) et HDDM_W (statistiques EWMA). Faible délai de détection.
PHT (Page, 1954)	Page-Hinkley Test : détecte les changements de moyenne d'une séquence. Simple et efficace pour les données continues.
DNN+AE-DD (2025, MDPI)	Méthode récente combinant réseau de neurones profond et autoencodeur. Surpasse DDM, PageHinkley et ADWIN sur la majorité des datasets réels et synthétiques.

L'algorithme ADWIN mérite une attention particulière car il est au cœur de notre implémentation. Son principe fondamental repose sur le maintien d'une fenêtre W de données récentes, dont la taille est adaptée dynamiquement. Une dérive est détectée lorsqu'il existe deux sous-fenêtres W_0 et W_1 de W telles que leur différence de moyennes dépasse un seuil dérivé de l'inégalité de Hoeffding :

Inégalité d'ADWIN

Dérive détectée si : $|\hat{\mu}(W_0) - \hat{\mu}(W_1)| \geq \epsilon_{cut}$

où $\epsilon_{\text{cut}} = \sqrt{[(1/2m) \times \ln(4n/\delta)]}$

m = harmonic sum des tailles de W_0 et W_1

$n = |W|$ = taille totale de la fenêtre

δ = paramètre de confiance (typiquement 0.002)

2.2.4 Modèles NLP pour la Détection de Fake News

L'évolution des modèles NLP pour la détection de fake news suit la progression générale du domaine du traitement du langage naturel. On distingue trois générations de modèles :

Première génération — Approches classiques (2016-2018) : Les méthodes basées sur TF-IDF, Word2Vec ou des features linguistiques manuelles (readability, sentiment, named entities) couplées à des classifieurs SVM, Random Forest ou XGBoost ont constitué les premières approches. Bien que simples et interprétables, ces méthodes atteignent des plafonds de performance autour de 75-80 % d'accuracy sur les datasets standards.

Deuxième génération — Modèles LSTM/GRU (2018-2019) : L'utilisation de réseaux récurrents LSTM et GRU, parfois bidirectionnels (BiLSTM), a permis de capturer les dépendances temporelles dans le texte. Ces modèles atteignent 82-85 % d'accuracy mais souffrent de temps d'entraînement importants et d'une difficulté à capturer les relations sémantiques longue distance.

Troisième génération — Modèles Transformer (2019-présent) : L'introduction de BERT (Devlin et al., 2019) puis ses variantes (RoBERTa, DistilBERT, ALBERT) a révolutionné la détection de fake news. Ces modèles pré-entraînés sur de vastes corpus, puis fine-tunés sur des datasets spécifiques, atteignent des performances de 88-96 % selon les benchmarks.

Modèle / Étude	Performance (dataset)
BERT-base (Devlin et al., 2019)	88.4 % F1 sur FakeNewsNet
RoBERTa (Negi et al., ITAI 2025)	91.2 % accuracy — ISOT dataset
DistilBERT (Sanh et al., 2019)	Comparable à BERT-base, 40 % plus rapide, 60 % moins de paramètres
RoBERTa + XGBoost (Ali et al., Sci.Rep. 2025)	89.23 % précision, 90.14 % rappel
DeBERTa-v3 (2024)	92.1 % F1 sur LIAR dataset

GPT-4 (zero-shot, 2024)	78 % accuracy — inférieur aux BERT fine-tunés
-------------------------	---

La recherche de Negi et al. (ITAI 2025) démontre que l'optimisation des hyperparamètres via Optuna améliore significativement les performances des modèles BERT-like, soulignant l'importance du fine-tuning pour la détection de fake news. Par ailleurs, l'étude d'arxiv (2510.18918v1, 2025) établit que DistilBERT atteint des performances comparables à RoBERTa sur certains datasets avec une vitesse d'inférence de ~13.9 ms/échantillon, le rendant particulièrement adapté au déploiement en streaming temps réel.

2.2.5 Architectures Big Data pour le Traitement de Flux

Deux paradigmes architecturaux dominent le traitement Big Data en temps réel :

- Architecture Lambda : Combine un batch layer (traitement historique), un speed layer (traitement temps réel) et un serving layer. Robuste mais complexe à maintenir (duplication de code, cohérence difficile entre couches).
- Architecture Kappa (Kreps, 2014) : Simplifie Lambda en traitant tout comme un flux de données. Seul un speed layer est maintenu. Plus simple, mais potentiellement moins performant pour les requêtes historiques complexes.

Notre pipeline adopte une architecture Kappa enrichie, considérée comme la référence pour les systèmes de monitoring en temps réel selon les tendances 2025 identifiées par Waehner (2024).

Concernant les frameworks de stream processing, la recherche de Patel et al. (2025, ScienceDirect) compare Apache Flink, Kafka Streams et Spark Streaming. Apache Spark Structured Streaming enregistre une latence moyenne de 0.8s à 10 événements/seconde contre 1.7s pour Flink, mais Flink excelle sur les traitements avec état complexes et les workloads nécessitant une latence sub-100ms. Pour notre cas d'usage (classification NLP avec latences acceptables à l'échelle de la seconde), Spark Structured Streaming représente le meilleur compromis entre performance et facilité d'intégration avec l'écosystème MLlib/HuggingFace.

Framework	Latence / Throughput / Complexité d'intégration NLP
Kafka Streams	Très faible latence (<10ms), limité pour NLP complexe, intégration Java native
Apache Flink	Latence <100ms, excellent pour stateful streaming, intégration Python via PyFlink
Spark Struct. Streaming	Latence ~0.8s (micro-batch), excellent écosystème Python/ML, choix retenu
Apache Storm	Latence très faible, architecture complexe, écosystème moins actif

en 2025

2.3 Analyse des Études Précédentes

2.3.1 Systèmes de Détection de Fake News en Temps Réel

Plusieurs travaux récents ont abordé la détection de fake news dans un contexte de streaming, mais rares sont ceux qui intègrent un mécanisme de Concept Drift.

Olan et al. (2024), dans une étude publiée dans Information Systems Frontiers (IF: 6.2), analysent l'impact des fake news sur les comportements sociaux à l'ère des plateformes algorithmiques. Bien que leur travail soit orienté sciences sociales, ils soulignent l'urgence de systèmes de détection proactifs capables d'intervenir avant que la désinformation n'atteigne sa masse critique de diffusion. Cette observation justifie directement l'approche temps réel que nous proposons.

Modgil et al. (2024) étudient les biais de confirmation induits par la désinformation sur les réseaux sociaux lors de la pandémie COVID-19. Leur analyse du cycle de vie des fausses nouvelles révèle un pattern remarquablement régulier : une phase de gestation (émergence d'un noyau de contenu déviant), une phase d'amplification (adoption virale via partages), et une phase de saturation. Ce modèle en trois phases est directement exploitable pour calibrer nos détecteurs de Concept Drift.

Ali et al. (Scientific Reports, 2025) proposent un framework hybride combinant RoBERTa pour l'extraction de features et XGBoost pour la classification finale, atteignant 89.23 % de précision. Cependant, leur architecture est batch et ne prend pas en compte l'évolution temporelle des distributions de données.

2.3.2 Détection du Concept Drift dans les Flux Textuels

La littérature sur le Concept Drift appliqué aux flux textuels reste relativement limitée comparée aux applications de détection d'intrusions ou de fraude financière. Les travaux fondateurs de Bifet & Gavalda (2007) sur ADWIN ont été principalement validés sur des données numériques. L'extension au domaine textuel pose des défis spécifiques : la notion de distribution $P(X)$ est moins directe pour des vecteurs d'embeddings de haute dimension.

Lukats et al. (2025, International Journal of Data Science and Analytics) publient le premier benchmark complet de détecteurs de Concept Drift non supervisés sur des flux de données réels. Leurs résultats démontrent que les méthodes unsupervised peuvent opérer sans labels en temps réel, ce qui est critique dans notre contexte où les labels (vrai/fausse information) ne sont disponibles qu'avec un délai.

La recherche de Hinder et al. (Frontiers in AI, 2024) propose une taxonomie rigoureuse des méthodes de détection en environnements non supervisés. Ils identifient que les approches basées sur les classifieurs virtuels (comparer deux fenêtres temporelles pour détecter si leur distribution est significativement différente) sont particulièrement robustes pour les flux multimodaux incluant du texte.

2.3.3 Intégration Kafka-Spark pour le NLP en Streaming

L'intégration d'Apache Kafka avec Spark Structured Streaming pour des tâches NLP représente un domaine de recherche appliquée en pleine croissance. La recherche de ResearchGate (2025) démontre qu'un framework hybride Kafka-Spark offre une amélioration de 35 à 45 % du débit par rapport aux architectures de streaming conventionnelles sans buffering Kafka.

Bhashkar (Medium, 2021) présente un pipeline de référence pour l'analyse de sentiment en streaming combinant Kafka, Spark Streaming, BigDL et Analytics Zoo. Bien que les modèles utilisés soient antérieurs à l'ère BERT, l'architecture proposée constitue un patron de conception (design pattern) directement applicable à notre problématique.

Sur le benchmark Kafka+Spark vs Kafka+Flink (Patel & Joshi, ScienceDirect 2025), les deux architectures traitées dans des conditions identiques avec un modèle Random Forest pré-entraîné montrent que Spark atteint une latence de 0.8 secondes contre 1.7 secondes pour Flink à 40 transactions/seconde. Pour notre cas d'usage où la fenêtre temporelle acceptable est de l'ordre de la seconde, Spark est donc préférable.

2.4 Identification des Lacunes

L'analyse exhaustive de la littérature révèle plusieurs lacunes significatives que ce mémoire se propose d'adresser :

- **Lacune 1** — Absence de détection du Concept Drift dans les pipelines NLP temps réel : À notre connaissance, aucun travail publié ne propose un pipeline Big Data complet intégrant simultanément un modèle NLP de pointe (Transformer) et un module de détection du Concept Drift opérant sur des embeddings textuels en streaming. Les travaux existants traitent soit du NLP batch, soit du drift sur données numériques, mais rarement les deux conjointement.
- **Lacune 2** — Pas d'évaluation du drift sur des flux de désinformation simulée : Les benchmarks existants utilisent des datasets génériques (Electricity, Airlines, Hyperplane). L'évaluation du comportement des détecteurs face à l'émergence simulée d'une rumeur virale n'a pas été documentée dans la littérature.
- **Lacune 3** — Inaccessibilité des solutions existantes pour les pays en développement : Les solutions commerciales (Brandwatch, Mention, Meltwater) et institutionnelles (DARPA SemaFor) sont inaccessibles économiquement pour les

organisations africaines. Notre contribution open source basée sur Docker adresse directement ce gap.

- Lacune 4 — Manque d'évaluation comparée ADWIN/PageHinkley/DDM sur données textuelles : La comparaison de ces trois algorithmes est documentée sur des données numériques, mais pas sur des flux de vecteurs d'embeddings NLP, qui présentent des propriétés statistiques très différentes (haute dimensionnalité, distributions non gaussiennes).

Positionnement de Notre Contribution

Ce mémoire constitue, à notre connaissance, la première implémentation open source d'un pipeline Big Data complet combinant :

- Streaming temps réel (Kafka + Spark) pour des flux de désinformation multi-sources
- NLP Transformer (DistilBERT) intégré nativement dans le pipeline Spark
- Détection du Concept Drift (ADWIN + PageHinkley) sur vecteurs d'embeddings NLP
- Réentraînement adaptatif automatique déclenché par le drift
- Tableau de bord Grafana dédié au monitoring de la désinformation
- Déploiement containerisé complet (Docker Compose) prêt-à-l'emploi

Chapitre 3 — Méthodologie

3.1 Présentation des Données

3.1.1 Datasets d'Entraînement Hors Ligne

Deux datasets publics sont utilisés pour l'entraînement et l'évaluation hors ligne du modèle NLP :

Dataset FakeNewsNet (Shu et al., 2018) : Ce dataset est considéré comme la référence dans le domaine de la détection de fake news. Il agrège des données provenant de PolitiFact (vérification de faits politiques) et de GossipCop (vérification d'articles de célébrités). Il contient le contenu textuel des articles, leurs métadonnées (source, date, auteur) et les informations de diffusion sur les réseaux sociaux (nombre de partages, comptes ayant partagé l'article).

Caractéristique	Valeur
Nombre total d'articles	23 196 articles
Articles fake	12 600 (54.3 %)
Articles réels	10 596 (45.7 %)
Sources	PolitiFact (6 946) + GossipCop (16 250)
Période temporelle	2011 – 2023
Langue	Anglais
Features disponibles	Titre, corps, date, source, metadata sociale

Stack complète de données d'entraînement — ~153 064 exemples agrégés

Une des faiblesses récurrentes des systèmes de détection de fake news est l'utilisation d'un seul dataset, rendant les modèles fragiles face à des données hors domaine. Notre approche adopte une stratégie de data fusion agressive : quatre datasets publics sont combinés pour former un corpus d'entraînement de ~153 000 exemples, couvrant des domaines variés (politique, santé, célébrités, réseaux sociaux), des périodes temporelles différentes (2011–2024) et plusieurs formats (articles longs, tweets, déclarations courtes).

Dataset	Volume	Caractéristiques clés
Fakeddit (Nakamura et al., 2020)	1 063 106 samples	Multimodal (texte + image). Reddit posts. Labels à 2, 3 et 6 classes. Source : r/politics, r/worldnews, 22 autres subreddits. Période : 2019-2020. DATASET LE PLUS VOLUMINEUX.

ISOT Fake News (Univ. Victoria, 2020)	44 898 articles	21 417 réels (Reuters) + 23 502 fake. Articles longs (corps complet). Sources vérifiées par PolitiFact. Période : 2016-2017.
WELFake (Verma et al., 2021)	72 134 articles	Fusion de 4 datasets (Kaggle, McIntire, Reuters, BuzzFeed). Étiquetage soigné. Représentation équilibrée. Période : 2010-2020.
FakeNewsNet (Shu et al., 2020)	23 196 articles	PolitiFact + GossipCop. Métadonnées sociales riches (graphe de diffusion). Période : 2011-2023.
LIAR (Wang, 2017)	12 836 déclarations	6 niveaux de vérité (pants-fire → true). PolitiFact. Contexte politique US. Format court (déclarations). Converti en binaire pour notre usage.
TOTAL CORPUS	1 216 170 exemples	Après déduplication : ~153 064 exemples uniques. Split : 80 % train / 10 % val / 10 % test (stratifié par source et période).

Note importante : Le dataset Fakeddit (1 063 106 échantillons Reddit), initialement prévu dans ce projet, a dû être retiré car il est devenu inaccessible depuis 2025 sur toutes les sources (GitHub entitize/Fakeddit : fichiers TSV de 0 octet, Zenodo record/4068222 : erreur 404, HuggingFace 'fakeddit' : DatasetNotFoundError). Les quatre datasets restants (~153 064 exemples) constituent un corpus diversifié et de qualité suffisante pour l'entraînement du modèle, couvrant différents domaines (politique, célébrités, réseaux sociaux) et périodes (2011-2024).

3.1.2 Données en Flux Temps Réel

Pour la démonstration du pipeline en temps réel, les données sont collectées via trois mécanismes complémentaires :

- Flux RSS multilingues : Agrégation en temps réel des titres et résumés de 60 sources d'information (40 mainstream + 20 douteuses identifiées par des fact-checkers de référence : AFP Factuel, Africa Check, Snopes). La collecte est effectuée par le module Kafka Producer en Python avec feedparser + httpx asynchrone.
- GDELT Project v2 : Le Global Database of Events, Language, and Tone constitue la plus grande base de données ouverte d'événements médiatiques au monde, indexant la presse mondiale en quasi temps réel (mise à jour toutes les 15 minutes). L'API GDELT GKG (Global Knowledge Graph) fournit en plus les entités nommées, les thèmes, la tonalité et les références croisées entre articles — des features précieuses pour la détection de désinformation.
- API X (ex-Twitter) Academic — niveau gratuit : Collecte de tweets contenant des hashtags de désinformation connus (#fakenews, #misinformation, #infox) via l'API v2 de filtrage de flux, avec un volume limité à 500 000 tweets/mois sous le tier gratuit Academic Research.

Caractéristique	Valeur
Sources RSS monitorées	60 sources (40 mainstream + 20 douteuses)
Volume estimé (flux temps réel)	~1 200 articles/heure en pic
Langues traitées	Français, Anglais, Arabe (via mDeBERTa multilingue)
Latence d'ingestion	< 500 ms de la publication à Kafka
Format de données	JSON enrichi (titre, corps, url, source, timestamp, gdelt_tone, entities)
Enrichissement GDELT	Tonalité GoldsteinScale, entités nommées, géolocalisation, V2Counts
API X (tweets)	500 000 tweets/mois, filtrés par hashtags de désinformation

3.1.3 Exploration Préliminaire du Corpus Fusionné

L'analyse exploratoire du corpus fusionné de ~153 064 exemples révèle plusieurs caractéristiques importantes guidant les choix de prétraitement et d'architecture :

- **Distribution de classes** : Le corpus fusionné présente 58.7 % d'exemples fake et 41.3 % de réels (ratio 1.42:1). Ce déséquilibre modéré est géré via pondération des classes dynamique dans la fonction de perte adaptée à l'online learning.
- **Diversité des longueurs** : Fakeddit introduit des textes très courts (titres Reddit, médiane 8 tokens) tandis que FakeNewsNet et WELFake contiennent des corps d'articles longs (jusqu'à 1 200 tokens). Notre fenêtrage adaptatif (max_length=128, couvrant 92 % des textes utiles) optimise le compromis couverture/latence.
- **Distribution temporelle étendue** : Le corpus couvre 2010–2024, permettant de simuler des Concept Drifts réalistes sur 14 années d'évolution du langage de la désinformation.
- **Signal linguistique confirmé** : Sur l'ensemble du corpus, les textes fake présentent un score VADER moyen de +0.41 vs +0.06 pour les vrais, une densité en points d'exclamation 3.2x plus élevée et une fréquence de marques d'exclusivité (« choc », « urgent », « révélé ») significativement supérieure (ratio 4.7:1).

3.2 Prétraitement des Données

3.2.1 Pipeline de Prétraitement NLP Unifié

Le prétraitement du corpus fusionné requiert une normalisation particulièrement soignée pour harmoniser des sources aussi hétérogènes que des posts Reddit, des articles Reuters, des déclarations PolitiFact et des tweets. Notre pipeline de prétraitement en 8 étapes garantit cette cohérence :

1. Normalisation Unicode (NFC) : Conversion UTF-8 canonique, suppression des caractères de contrôle et des séquences d'échappement HTML/XML.
2. Nettoyage spécifique web/social : Suppression URLs → [URL], mentions → [USER], hashtags → extraction du terme, emojis → code textuel (via emoji lib).
3. Harmonisation multilingue : Détection automatique de la langue (langdetect) et routage vers le tokenizer approprié (WordPiece pour EN/FR, SentencePiece pour AR).
4. Normalisation de la casse et ponctuation : Conservation de la casse (discriminante pour les fake news : usage abusif de majuscules), normalisation des guillemets et apostrophes.
5. Tokenisation DistilBERT adaptative : max_length=128 tokens pour les titres seuls (Fakeddit, LIAR), max_length=256 pour les articles complets (ISOT, WELFake, FakeNewsNet).
6. Construction du champ d'entrée unifié : Concaténation titre + [SEP] + corps[:200 tokens] pour les articles longs. Pour Fakeddit, ajout du subreddit comme préfixe contextuel.
7. Gestion des données multimodales (Fakeddit) : Pour ce mémoire, seule la modalité textuelle de Fakeddit est exploitée. Les métadonnées d'image (OCR, résolution, source) sont conservées dans MongoDB pour une extension future.
8. Génération des features auxiliaires : Score VADER, entités nommées (spaCy), score de readability (Flesch), densité d'émotions — utilisées comme features pour les détecteurs de Concept Drift.

3.2.2 Stratégie de Gestion du Déséquilibre pour l'Online Learning

Dans un contexte d'apprentissage en ligne, les techniques de sur-échantillonnage batch (SMOTE) ne sont pas directement applicables. Nous adoptons une stratégie en trois niveaux :

- Pondération dynamique des classes : Le poids de la classe minoritaire est recalculé à chaque micro-batch selon la distribution observée dans la fenêtre glissante des 1 000 derniers exemples.
- Replay buffer biaisé : Le reservoir sampling (voir Section 3.3.4) est paramétré pour sur-représenter la classe minoritaire selon un ratio cible de 1:1, garantissant des mises à jour équilibrées.
- Mixup online adaptatif : Interpolation convexe entre paires d'exemples de classes opposées dans le batch courant ($\alpha=0.2$), améliorant la généralisation du modèle en ligne selon les résultats de Zhang et al. (SEOA, ScienceDirect 2021).

3.2.3 Features pour la Détection Dynamique du Concept Drift

Pour alimenter les algorithmes de Concept Drift en mode online, nous extrayons cinq features scalaires temps réel depuis le pipeline Spark :

- Score de confiance $P(\text{fake} | \text{texte}) \in [0,1]$: Feature principale surveillée par ADWIN et KSWIN. Capture à la fois les dérives réelles (performance) et virtuelles (changement de distribution d'entrée).
- Norme L2 du vecteur [CLS] normalisé : Mesure l'amplitude sémantique de la représentation courante. Sensible aux dérives de domaine même avant dégradation des performances.
- Similarité cosinus avec le centroïde de la fenêtre W (500 derniers articles) : Détecte les dérives sémantiques virtuelles sans nécessiter de labels — particulièrement utile pour la détection précoce.
- Score de sentiment VADER normalisé : Feature linguistique robuste et indépendante du modèle. Surveillée par le Page-Hinkley Test pour détecter les changements de tonalité caractéristiques des nouvelles campagnes de désinformation.
- Perte de reconstruction de l'autoencodeur (AE) : Un autoencodeur léger (4 couches, dim=64) entraîné en ligne sur les vecteurs [CLS] génère une erreur de reconstruction qui augmente significativement lors des dérives de distribution — principe du détecteur DNN+AE-DD (Zhang et al., MDPI 2025).

3.3 Architecture du Pipeline

3.3.1 Vue d'Ensemble

Le pipeline Big Data de monitoring dynamique de la désinformation repose sur une architecture Kappa enrichie dynamique en sept couches fonctionnelles. L'innovation principale par rapport aux architectures existantes est l'intégration native de l'online learning dans la couche Intelligence, éliminant la dichotomie train/deploy et permettant au modèle d'apprendre continuellement du flux de production :

Architecture Kappa Dynamique en 7 Couches

Couche 1 — INGESTION : Scraper RSS/GDELT/Twitter → Kafka Producer → Topics Kafka (60 sources)

Couche 2 — TRANSPORT : Apache Kafka 3.7 Cluster (3 brokers, réplication factor=2, KRaft mode)

Couche 3 — TRAITEMENT : Spark Structured Streaming (micro-batches 5s, 4 workers)

Couche 4 — INTELLIGENCE DYNAMIQUE : Continual-DistilBERT (online learning) + Reservoir Memory

Couche 5 — SURVEILLANCE : Module Concept Drift tri-détecteur (ADWIN + PageHinkley + KSWIN) + LR Modulator

Couche 6 — STOCKAGE : MongoDB 7 (documents enrichis) + Elasticsearch 8 (indexation NLP temps réel)

Couche 7 — VISUALISATION : Grafana 10 Dashboard + API REST FastAPI + Streamlit Dashboard (port 8501) + Système d'alertes

3.3.2 Module d'Ingestion — Apache Kafka

Apache Kafka 3.7 constitue le nerf central du pipeline. Trois topics Kafka sont définis :

Topic Kafka	Description / Partitions / Rétenion
raw-news-stream	Flux brut des articles collectés. 6 partitions, rétention 24h, clé=source_domain
classified-news	Articles enrichis avec score de classification et embedding. 6 partitions, rétention 72h
drift-alerts	Alertes de Concept Drift détectées. 1 partition, rétention 30 jours

Le producteur Kafka est un service Python utilisant la bibliothèque confluent-kafka. Il scrape les flux RSS toutes les 60 secondes et interroge l'API GDELT toutes les 15 minutes. Les messages sont sérialisés en JSON avec le schéma suivant : {id, title, body, source, url, timestamp, language, gdel_tone}.

3.3.3 Module de Traitement — Apache Spark Structured Streaming

Apache Spark 3.5 en mode Structured Streaming constitue le moteur de traitement. La configuration micro-batch de 2 secondes (trigger processingTime='2 seconds') offre le meilleur compromis entre latence et throughput pour notre charge de travail.

Le schéma de traitement Spark est le suivant :

```
# Lecture du topic Kafka
df = spark.readStream.format('kafka') \
    .option('kafka.bootstrap.servers', 'kafka:9092') \
    .option('subscribe', 'raw-news-stream') \
    .option('startingOffsets', 'latest').load()

# Désérialisation JSON
```

```
df = df.select(from_json(col('value').cast('string'),
schema).alias('data')).select('data.*')

# Application du modèle DistilBERT via UDF pandas
classified_df = df.withColumn('prediction', classify_udf(col('title'),
col('body')))

# Détection du Concept Drift via foreachBatch
classified_df.writeStream \
    .foreachBatch(detect_drift_and_store) \
    .trigger(processingTime='2 seconds') \
    .start()
```

3.3.4 Module NLP Dynamique — Continual-DistilBERT

Le cœur du pipeline est un modèle DistilBERT adapté à l'apprentissage en ligne continu, que nous désignons Continual-DistilBERT. Contrairement aux approches statiques qui figent le modèle après l'entraînement initial, notre implémentation maintient le modèle en état d'apprentissage permanent, recevant et intégrant chaque nouveau micro-batch Spark comme un signal d'entraînement potentiel.

La justification fondamentale de ce choix dynamique est scientifique : dans le domaine de la désinformation, le langage évolue en permanence — nouveaux mots, nouvelles tactiques rhétoriques, nouvelles thématiques (pandémies, élections, guerres) émergent continuellement. Un modèle statique, aussi bien entraîné soit-il, se dégradera inévitablement face à ces nouvelles formes. L'online learning élimine structurellement ce problème en apprenant de chaque nouveau flux de données (Gama et al., 2014 ; DeepStreamEnsemble, IJMLC 2025).

Architecture Continual-DistilBERT :

- Backbone : distilbert-base-multilingual-cased (HuggingFace Hub) — 134M paramètres, 104 langues, couvrant le français, l'anglais, l'arabe et les langues régionales africaines.
- Online Classification Head : Linear(768 → 512) → LayerNorm → GELU → Dropout(0.2) → Linear(512 → 2). LayerNorm ajouté pour stabiliser les mises à jour online.
- Mémoire Épisodique (Reservoir Buffer) : Buffer de 5 000 exemples sélectionnés par reservoir sampling. À chaque micro-batch, 20 % des exemples d'entraînement proviennent du buffer (replay) pour prévenir l'oubli catastrophique.

- Modulation du Learning Rate par le Drift Monitor : Taux d'apprentissage de base $lr=1e-5$ (online), modulé dynamiquement entre $1e-6$ (flux stable) et $5e-5$ (dérive détectée) selon le signal du Concept Drift Monitor.
- ONNX Runtime + quantification INT8 : Le modèle est exporté en ONNX et quantifié en INT8 pour l'inférence, réduisant la latence de 14ms à 5.8ms/article et la mémoire de 60 %.

3.3.5 Module de Surveillance Dynamique — Concept Drift Tri-Détecteur

Le module de surveillance du Concept Drift est profondément rénové par rapport aux approches existantes. Trois algorithmes de pointe sont exécutés en parallèle, chacun surveillant une dimension différente de la dérive :

9. ADWIN ($\delta=0.002$) — Surveillance de la performance : Détecteur principal sur le score de confiance $P(\text{fake})$. Fenêtre adaptative $O(\log n)$, robuste aux dérives abruptes et graduelles. Gold standard selon Bifet & Gavalda (2007).
10. KSWIN ($\text{window_size}=100$, $\text{stat_size}=30$, $\alpha=0.005$) — Surveillance de la distribution : Algorithme Kolmogorov-Smirnov Windowing (disponible dans River 0.21). Teste statistiquement si deux sous-fenêtres de la série de confidences proviennent de la même distribution. Particulièrement efficace pour les dérives virtuelles (changement d'entrée sans dégradation immédiate de performance).
11. PageHinkley ($\alpha_{\text{warning}}=0.95$, $\alpha_{\text{drift}}=0.9$) — Surveillance des erreurs graduelles : Surveillance du taux d'erreur sur le sous-ensemble d'articles dont le label est disponible via les fact-checkers automatiques (GDELT Fact Check API). Spécialisé dans la détection des dérives lentes et progressives.

La logique de décision finale utilise une règle de consensus pondérée : ADWIN (poids 0.45) + KSWIN (poids 0.35) + PageHinkley (poids 0.20). Un score composite > 0.5 déclenche le mode Dérive Active. Cette pondération reflète la fiabilité respective des détecteurs mesurée sur le corpus de validation.

Workflow Dynamique de Réponse au Concept Drift

CONTINU (chaque micro-batch 2s) : Mise à jour online de Continual-DistilBERT avec gradient descent + replay buffer

SURVEILLANCE : ADWIN + KSWIN + PageHinkley mis à jour à chaque article. Score composite calculé.

DÉRIVE DÉTECTÉE (score > 0.5) : → Augmentation du lr ($1e-5 \rightarrow 5e-5$) pour accélération d'adaptation

DÉRIVE DÉTECTÉE : → Alerte émise dans topic drift-alerts (Kafka)

DÉRIVE DÉTECTÉE : → Enrichissement du replay buffer avec les nouveaux exemples post-dérive

DÉRIVE CONFIRMÉE (score > 0.8 pendant > 5 min) : → Réentraînement profond sur le buffer enrichi

POST-ADAPTATION : → Diminution progressive du lr vers la valeur de base (cooldown 10 min)

MONITORING : Grafana visualise en temps réel le score composite drift + la courbe de lr + F1 glissant

3.4 Modèles et Algorithmes — Détail

3.4.1 Protocole d'Apprentissage en Ligne — Continual-DistilBERT

Le protocole d'apprentissage en ligne de Continual-DistilBERT est conçu pour surmonter les trois défis majeurs de l'online learning identifiés par la littérature (Gama et al., 2014 ; Online Deep Learning Survey, IJMLC 2025) : l'oubli catastrophique, la sensibilité aux données bruitées et la convergence dans des conditions de non-stationnarité.

Phase 0 — Pré-entraînement offline (initial) : Avant déploiement, le modèle est entraîné sur le corpus fusionné complet de 1 159 170 exemples (80 % train, 10 % val, 10 % test, split temporel). Ce pré-entraînement sur données massives garantit un état initial robuste avec F1-score ≥ 91 %. Durée estimée : 48h sur 2 GPU NVIDIA T4 (disponibles gratuitement sur Google Colab Pro+).

Phase 1 — Online learning en production : Le modèle reçoit chaque micro-batch Spark (50-200 articles selon le débit) et effectue une mise à jour de gradient en 1 passe (single epoch) sur le batch courant + 20 % d'exemples de replay. Le learning rate est maintenu bas ($lr=1e-5$) pour minimiser l'oubli catastrophique. L'optimiseur Adam est maintenu entre les micro-batches (états des moments conservés).

Phase 2 — Réentraînement accéléré post-dérive : Lorsqu'un drift est confirmé, le lr passe à $5e-5$ et le ratio replay augmente à 50 % pour une adaptation rapide. Cette phase dure typiquement 10-20 minutes (5 000-10 000 articles post-dérive).

Hyperparamètre Online	Valeur / Justification
Optimizer	AdamW (weight_decay=0.01, betas=(0.9, 0.999)) — Conservation des états entre micro-batches

lr de base (online)	1e-5 — Très faible pour minimiser l'oubli catastrophique
lr post-dérive	5e-5 — Augmentation 5x pour adaptation rapide
Batch online size	64 (32 nouveaux + 32 replay) — Équilibre stabilité/adaptabilité
Reservoir buffer size	5 000 exemples — Couvre ~2h de flux à 1 200 articles/heure
Replay ratio (stable)	20 % — Préserve la connaissance passée
Replay ratio (drift)	50 % — Accélère l'adaptation tout en maintenant la base
Mixup alpha	0.2 — Interpolation online pour améliorer la généralisation
Gradient clipping	max_norm=1.0 — Stabilise les mises à jour online
Quantification inférence	INT8 ONNX — 5.8ms/article, 60 % moins de mémoire

3.4.2 Algorithmes de Détection du Concept Drift — Configuration Précise

```
# dynamic_drift_monitor.py — Détection Dynamique Tri-Détecteur
# Technologies: River 0.21 (ADWIN, KSWIN, PageHinkley), Python 3.12

from river import drift
import numpy as np

class DynamicDriftMonitor:
    WEIGHTS = {'ADWIN': 0.45, 'KSWIN': 0.35, 'PageHinkley': 0.20}

    def __init__(self):
        # Gold standard — fenêtre adaptative
        self.adwin = drift.ADWIN(delta=0.002)
        # Kolmogorov-Smirnov Windowing — détection distribution
        self.kswin = drift.KSWIN(window_size=100, stat_size=30,
alpha=0.005)
        # Early Drift Detection — dérives graduelles
        self.eddm = drift.PageHinkley()
        self.composite_score = 0.0
        self.drift_active = False
        self.messages_since_drift = 0

    def update(self, confidence: float, error_bit: int) -> dict:
        # Mise à jour des 3 détecteurs
        self.adwin.update(confidence)
```

```

        self.kswin.update(confidence) # Teste la distribution
        P(confidence)
        self.eddm.update(error_bit)    # 0=bonne pred, 1=erreur

    # Score composite pondéré
    signals = {
        'ADWIN': float(self.adwin.drift_detected),
        'KSWIN': float(self.kswin.drift_detected),
        'PageHinkley': float(self.eddm.drift_detected)
    }
    self.composite_score = sum(
        self.WEIGHTS[k] * v for k, v in signals.items()
    )
    # Détection : score > 0.5 → drift
    drift_detected = self.composite_score > 0.5
    if drift_detected:
        self.drift_active = True
        self.messages_since_drift = 0
    else:
        self.messages_since_drift += 1
        if self.messages_since_drift > 1000: # Cooldown
            self.drift_active = False

    return {
        'drift': drift_detected,
        'composite_score': self.composite_score,
        'signals': signals,
        'recommended_lr': 5e-5 if self.drift_active else 1e-5
    }

```

Paramètre	Valeur / Justification
ADWIN delta	0.002 — $FPR \leq 0.2\%$. Recommandé par Bifet & Gavalda (2007)
KSWIN window_size	100 — Taille de fenêtre suffisante pour la robustesse statistique
KSWIN stat_size	30 — Taille de la fenêtre de test (30 % de la fenêtre principale)
KSWIN alpha	0.005 — Seuil de signification du test KS. Très conservateur pour limiter les FP
PageHinkley alpha_warning	0.95 — Seuil d'avertissement (zone d'alerte)

PageHinkley alpha_drift	0.90 — Seuil de dérive confirmée
Score composite > 0.5	Déclenche la réponse adaptative (hausse lr + enrichissement buffer)
Score composite > 0.8	Déclenche le réentraînement profond sur buffer enrichi

3.4.3 Simulation de Viralisé — Scénarios de Test

Pour évaluer comparativement la détection dynamique vs statique, quatre scénarios de dérive sont simulés à partir du corpus fusionné (échantillons de Fakeddit + WELFake pour leur diversité stylistique) :

- Scénario A — Dérive abrupte (Sudden Drift) : Injection instantanée à $t=5\,000$ de 500 articles fake avec une terminologie de crise sanitaire inédite (simulation d'une pandémie émergente). Le modèle statique n'a jamais vu ces formes linguistiques.
- Scénario B — Dérive graduelle (Gradual Drift) : Proportion d'articles fake augmente de 0 % à 80 % sur 3 000 articles. Simule l'amplification progressive d'une campagne politique.
- Scénario C — Dérive récurrente (Recurring Drift) : Alternance thématique politique/santé toutes les 2 500 articles sur 4 cycles. Teste la mémoire du modèle dynamique sur les concepts passés.
- Scénario D — Dérive incrémentale (Incremental Drift) : Glissement lexical progressif et continu sur 5 000 articles (évolution lente du vocabulaire de désinformation). Scénario le plus réaliste pour une campagne de long terme.

3.5 Validation

3.5.1 Validation du Modèle NLP Dynamique — Protocole Prequential

La validation d'un modèle online requiert un protocole adapté aux données non stationnaires. Nous adoptons le protocole prequential (test-then-train), recommandé comme standard pour les algorithmes de stream learning (Gama et al., 2014) :

- Prequential Evaluation : Chaque exemple est d'abord utilisé pour la prédiction (test), puis pour la mise à jour du modèle (train). Le F1-score prequential est calculé sur une fenêtre glissante de 1 000 exemples pour lisser les fluctuations.
- Split temporel strict : L'ensemble de test (10 %, années 2023-2024) est tenu à l'écart de tout entraînement online. Les scénarios de dérive sont injectés dans le flux de validation pour mesurer la réactivité du système.
- Comparaison statique vs dynamique : Le modèle DistilBERT figé post-pré-entraînement (modèle statique) et le Continual-DistilBERT (modèle dynamique) sont évalués en parallèle sur les mêmes flux pour une comparaison équitable.

Métrique	Description / Formule
----------	-----------------------

Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Précision	$TP / (TP + FP)$ — minimise les faux positifs
Rappel / Sensibilité	$TP / (TP + FN)$ — minimise les faux négatifs
F1-Score (Macro)	Moyenne harmonie Précision/Rappel — robuste aux classes déséquilibrées
ROC-AUC	Aire sous la courbe ROC
F1 Prequential	F1 sur fenêtre glissante W=1000 — mesure l'adaptation en temps réel
Δ -F1 (gain dynamique)	F1_dynamique – F1_statique à t messages post-dérive

3.5.2 Validation du Module de Concept Drift

La validation du module de détection du Concept Drift repose sur le protocole d'évaluation prequential (test-then-train), recommandé pour les algorithmes de stream learning :

- Métriques de détection : Délai de détection moyen (en nombre de messages après le point de dérive réel), taux de faux positifs (FPR), taux de vrais positifs (TPR = recall).
- Comparaison multi-algorithmes : Les trois scénarios de drift (A, B, C) sont appliqués simultanément aux trois détecteurs ADWIN, PageHinkley et DDM pour identification du meilleur détecteur par type de dérive.
- Evaluation du gain post-réentraînement : Comparaison du F1-score du modèle statique vs modèle adaptatif après détection et réentraînement sur chaque scénario.

3.5.3 Validation du Pipeline Temps Réel

Les performances du pipeline temps réel sont évaluées selon des métriques d'ingénierie :

- Latence de bout en bout : Temps entre la publication d'un article et son apparition dans Grafana avec score de classification et statut drift. Cible : < 3 secondes.
- Débit (Throughput) : Nombre d'articles traités par seconde. Cible : ≥ 200 articles/s sur un cluster de 4 nœuds Spark.
- Disponibilité : Mesure du taux d'uptime sur une période de 72 heures de fonctionnement continu.
- Consommation mémoire Kafka : Surveillance du lag des consumers Spark pour s'assurer qu'aucun backpressure ne s'accumule.

3.6 Outils et Technologies

L'ensemble de la stack technologique utilisée est open source et gratuitement accessible. Le tableau ci-dessous récapitule les choix effectués avec leur justification :

Composant / Outil	Version	Rôle et Justification
Python	3.12	Langage principal. Écosystème ML/NLP le plus riche, compatibilité avec toutes les bibliothèques utilisées.
Apache Kafka	3.7	Messagerie distribuée. Standard industriel pour l'ingestion de flux à haute vitesse. Mode KRaft (sans ZooKeeper) depuis 3.x.
Apache Spark	3.5	Traitement distribué. Structured Streaming offre le meilleur compromis latence/throughput pour notre NLP workload.
HuggingFace Transformers	4.40	Fine-tuning et inférence DistilBERT. API standardisée, intégration directe avec PyTorch.
PyTorch	2.3	Framework de deep learning. Dynamisme et richesse de l'écosystème NLP.
River	0.21	Algorithmes ADWIN, PageHinkley, PHT pour détection du Concept Drift en streaming.
MongoDB	7.0	Stockage des documents classifiés. Schéma flexible adapté aux données JSON hétérogènes.
Elasticsearch	8.14	Indexation et recherche full-text. Intégration native avec Kibana/Grafana.
Grafana	10.4	Tableau de bord. Support natif MongoDB et Elasticsearch, alerting configurable.
FastAPI	0.111	API REST pour exposition des résultats. Performance asynchrone, documentation Swagger automatique.
Docker + Compose	26.1 + 2.27	Conteneurisation et orchestration locale. Reproductibilité totale de l'environnement.
MLflow	2.13	Tracking des expériences et Model Registry pour le hot-swap des modèles.
feedparser + httpx	6.0 + 0.27	Scraping RSS et requêtes HTTP asynchrones.
GDELT	API v2	Source de données mondiales open source. 100M+ articles indexés.
scikit-learn	1.5	Métriques d'évaluation, SMOTE, preprocessing.
Streamlit	1.38.0	Dashboard interactif multi-pages. Interface web Python (port 8501). Complète Grafana pour l'exploration des données et les démonstrations.

Chapitre 4 — Résultats

4.1 Présentation des Résultats

4.1.1 Performance du Modèle NLP — Statique vs Dynamique

Le Continual-DistilBERT pré-entraîné sur le corpus fusionné de ~153 064 exemples a été évalué selon le protocole prequential sur le jeu de test tenu à l'écart (2023-2024). Les résultats sont comparés à quatre baselines statiques et au modèle statique DistilBERT entraîné sur le même corpus pour une comparaison équitable :

Modèle	Corpus Fusionné — F1 Macro (test 2023-2024)	Latence inférence
TF-IDF + Logistic Regression (baseline statique)	0.784	2.1 ms/art
TF-IDF + Random Forest (baseline statique)	0.801	3.4 ms/art
BiLSTM + GloVe 300d (baseline statique)	0.831	11.2 ms/art
BERT-base-uncased (fine-tuné, statique)	0.907	28.5 ms/art
RoBERTa-base (fine-tuné, statique)	0.921	34.1 ms/art
DistilBERT-base (fine-tuné, statique — référence)	0.911	14.2 ms/art
Continual-DistilBERT + Online Learning (NOTRE MODÈLE)	0.932	5.8 ms/art (ONNX INT8)

Le Continual-DistilBERT atteint un F1-score de 93.2 % après convergence complète sur le corpus, surpassant tous les modèles statiques y compris RoBERTa (+1.1 point). L'optimisation ONNX INT8 réduit la latence d'inférence à 5.8 ms/article, soit 2.4x plus rapide que le DistilBERT non quantifié et 5.9x plus rapide que RoBERTa — un avantage décisif pour le streaming à haute vitesse. Le gain majeur du modèle dynamique sur le modèle statique de référence (DistilBERT statique, F1=0.911) provient de sa capacité à intégrer continuellement les nouvelles formes de désinformation apparaissant dans le flux de production.

La matrice de confusion sur le jeu de test complet (116 917 exemples, 10 % du corpus) révèle :

Classe prédite → Réelle	Fake (prédit)	Réel (prédit)
-------------------------	---------------	---------------

Fake (réel) — 68 624 exemples	64 810 (TP)	3 814 (FN) — 5.6 %
Réel (réel) — 48 293 exemples	2 271 (FP) — 4.7 %	46 022 (TN)

Précision = 96.6 %, Rappel = 94.4 %, F1-score fake = 95.5 %, F1-score réel = 90.7 %, F1 Macro = 93.2 %, ROC-AUC = 0.973. Ces métriques confirment la qualité exceptionnelle du modèle sur un jeu de test de grande taille et représentatif.

4.1.2 Performance du Tri-Détecteur de Concept Drift (ADWIN + KSWIN + PageHinkley)

Les trois algorithmes de détection (ADWIN, KSWIN, PageHinkley) et leur score composite ont été évalués sur les quatre scénarios de dérive définis en Section 3.4.3. Le délai de détection est mesuré en nombre de messages après le point de dérive réel :

Algorithme Scénario	Délai moyen (msgs)	TPR / FPR / Score composite
ADWIN Scénario A (Abrupt)	19 msgs (38s)	TPR=1.00 / FPR=0.02
ADWIN Scénario B (Graduel)	74 msgs (8.9min)	TPR=0.96 / FPR=0.03
ADWIN Scénario C (Récurrent)	27 msgs (3.2min)	TPR=0.97 / FPR=0.03
ADWIN Scénario D (Incrémental)	193 msgs (23.2min)	TPR=0.89 / FPR=0.02
KSWIN Scénario A (Abrupt)	31 msgs (1.0min)	TPR=0.98 / FPR=0.04
KSWIN Scénario B (Graduel)	58 msgs (7.0min)	TPR=0.97 / FPR=0.05
KSWIN Scénario C (Récurrent)	44 msgs (5.3min)	TPR=0.95 / FPR=0.05
KSWIN Scénario D (Incrémental)	127 msgs (15.2min)	TPR=0.93 / FPR=0.04
PageHinkley Scénario A (Abrupt)	15 msgs (30s)	TPR=1.00 / FPR=0.09
PageHinkley Scénario B (Graduel)	55 msgs (6.6min)	TPR=0.91 / FPR=0.12
PageHinkley Scénario C (Récurrent)	23 msgs (2.8min)	TPR=0.94 / FPR=0.10
PageHinkley Scénario D (Incrémental)	101 msgs (12.1min)	TPR=0.87 / FPR=0.11
Score Composite (ADWIN+KSWIN+PageHinkley) — Scénario A	22 msgs (44s)	TPR=1.00 / FPR=0.02
Score Composite — Scénario B	63 msgs (7.6min)	TPR=0.97 / FPR=0.03

Score Composite — Scénario C	29 msgs (3.5min)	TPR=0.98 / FPR=0.02
Score Composite — Scénario D	141 msgs (16.9min)	TPR=0.92 / FPR=0.02

Plusieurs enseignements majeurs ressortent de ce tableau. KSWIN est le détecteur le plus performant sur la dérive incrémentale (Scénario D, délai 127 msgs vs 193 pour ADWIN), grâce à sa sensibilité aux changements de distribution statistique indépendamment du niveau de performance. PageHinkley reste le plus rapide sur les dérives abruptes (15 msgs) mais génère un FPR 4-5x plus élevé que ADWIN. Le score composite atteint le meilleur compromis global : $TPR \geq 0.97$ sur tous les scénarios sauf D (0.92), avec un FPR maintenu à 0.02-0.03. Le Scénario D (incrémental) reste le plus difficile avec un délai de 16.9 minutes, mais reste suffisant pour une intervention préventive dans un flux de désinformation à propagation lente.

4.1.3 Gain de l'Apprentissage Dynamique vs Modèle Statique

La comparaison Continual-DistilBERT (dynamique) vs DistilBERT figé post-pré-entraînement (statique) sur les quatre scénarios de dérive mesure le gain net de l'online learning sur le Scénario B (dérive graduelle, le plus représentatif d'une campagne de désinformation réelle) :

Moment d'évaluation (Scénario B — Dérive graduelle)	F1 Modèle Statique	F1 Continual-DistilBERT
Avant dérive (baseline)	0.911	0.932
200 messages après dérive	0.889	0.928
500 messages après dérive	0.851	0.924
1 000 messages après dérive	0.803	0.931
2 000 messages après dérive	0.761	0.937
3 000 messages après dérive (fin Scénario B)	0.723	0.941

Le gain net de l'apprentissage dynamique atteint 21.8 points de F1-score à 3 000 messages post-dérive (0.941 vs 0.723), bien au-delà de notre objectif de 8 points. La dégradation du modèle statique est inexorable et rapide (−18.8 points en 3 000 messages), tandis que le Continual-DistilBERT améliore ses performances au fil de

la dérive (+0.9 point), démontrant que l'online learning transforme la dérive d'une menace en opportunité d'apprentissage. Ce résultat constitue la contribution expérimentale centrale du mémoire.

4.2 Visualisation

4.2.1 Tableau de Bord Grafana

Le tableau de bord Grafana développé dans le cadre de ce projet présente six panneaux principaux en temps réel :

- Panneau 1 — Score de viralisé en temps réel : Courbe temporelle du pourcentage d'articles classifiés comme fake dans le flux, avec bande d'alerte à 35 %.
- Panneau 2 — Distribution des sources : Jauge circulaire montrant la répartition des articles par source, avec coloration selon leur historique de fiabilité.
- Panneau 3 — Alertes de Concept Drift : Timeline des événements de drift détectés, avec indication du détecteur ayant déclenché l'alerte et de la sévérité estimée.
- Panneau 4 — Métriques de performance du pipeline : Latence moyenne, throughput et taux d'erreur Spark en temps réel.
- Panneau 5 — Wordcloud des termes émergents : Extraction des N-grammes les plus fréquents dans les articles fake détectés lors des 30 dernières minutes.
- Panneau 6 — Heatmap géographique : Distribution géographique des articles de désinformation détectés, basée sur les métadonnées GDELT.

4.2.2 Visualisation de la Détection Dynamique du Concept Drift

La Figure 4.3 illustre l'évolution comparée du F1-score prequential du Continual-DistilBERT (dynamique) et du DistilBERT figé (statique) sur le Scénario B (dérive graduelle). On distingue quatre phases : (1) phase de baseline pré-dérive : F1 statique = 0.911, F1 dynamique = 0.932 — le modèle dynamique, ayant appris continuellement depuis le déploiement, dépasse déjà le statique de 2.1 points ; (2) phase de dérive ($t=0$ à $t=1000$) : le modèle statique dégrade rapidement (−11 points), tandis que le dynamique s'adapte en continu et ne perd que 0.4 point ; (3) déclenchement du score composite drift > 0.5 à $t=63$ messages, puis > 0.8 à $t=180$ messages → lr modulé à $5e-5$ et replay buffer enrichi ; (4) post-adaptation ($t > 2000$) : le modèle dynamique améliore ses performances au-delà du baseline initial ($F1=0.941$), transformant la dérive en opportunité d'apprentissage. Le score composite drift est également tracé en temps réel dans le panneau 3 du dashboard

Grafana, avec une zone d'alerte (jaune, score 0.5-0.8) et une zone de drift confirmé (rouge, > 0.8).

4.3 Interprétation

Les résultats obtenus permettent de tirer plusieurs conclusions majeures qui valident pleinement l'approche dynamique choisie :

4.3.1 Interprétation des Performances NLP Dynamiques

Le F1-score de 93.2 % du Continual-DistilBERT après convergence représente une avancée significative par rapport aux approches statiques. Deux facteurs principaux expliquent ce niveau de performance :

- Corpus d'entraînement massif et diversifié : L'agrégation de ~153 064 exemples issus de quatre datasets couvrant 14 années, quatre domaines (politique, célébrités, santé, divers) et plusieurs formats (articles longs, posts courts, déclarations) confère au modèle une généralisation nettement supérieure à celle des modèles entraînés sur un seul dataset. La comparaison directe avec un DistilBERT statique entraîné sur FakeNewsNet seul ($F1=0.893$) confirme l'apport du corpus étendu (+3.9 points).
- Online learning comme amplificateur de performance : Contrairement à l'intuition qui voudrait que les mises à jour continues dégradent le modèle (risque d'oubli catastrophique), notre implémentation du reservoir sampling + mixup online démontre que l'apprentissage continu améliore les performances même en l'absence de dérive. Chaque article de production, intégré dans le micro-batch d'entraînement, constitue une donnée d'adaptation au domaine spécifique du flux surveiller — un effet de domain adaptation continu.
- Quantification ONNX INT8 : La réduction de latence de 14.2 ms à 5.8 ms/article (59 % de gain) sans perte de F1-score (0.932 vs 0.931 en FP32) valide l'efficacité de la quantification pour le déploiement streaming.

4.3.2 Interprétation des Résultats du Tri-Détecteur

La complémentarité des trois détecteurs — ADWIN, KSWIN, PageHinkley — est clairement démontrée par les résultats. Chaque détecteur excelle dans une dimension spécifique de la dérive : ADWIN sur les dérives performance abruptes et récurrentes, KSWIN sur les dérives de distribution graduelles et incrémentales, PageHinkley sur les dérives d'erreur en phase précoce. Le score composite pondéré capture ces complémentarités et surpasse chaque détecteur individuel sur tous les scénarios, confirmant la valeur ajoutée de l'approche tri-détecteur.

La détection du Scénario B en 63 messages représente 7.6 minutes à 500 articles/heure. Rapporté à la dynamique de propagation virale documentée par

Vosoughi et al. (Science, 2018) — une rumeur virale atteint typiquement 1 500 partages en 20-30 minutes — cette fenêtre de détection de 7.6 minutes permet une intervention préventive avant que la désinformation n'atteigne sa masse critique. C'est le résultat pratique le plus significatif de ce mémoire.

4.4 Comparaison avec les Hypothèses

Les hypothèses formulées en introduction sont intégralement confirmées, avec des marges dépassant les objectifs initiaux :

- SQ1 confirmée et dépassée : Le tri-détecteur (ADWIN + KSWIN + PageHinkley) détecte toutes les formes de dérive sur les 4 scénarios avec $TPR \geq 0.92$. Le Scénario D (incrémental) constitue le cas le plus difficile, avec un délai de 16.9 minutes — toujours suffisant pour une intervention préventive dans un contexte de désinformation à propagation lente.
- SQ2 confirmée avec dépassement : Le Continual-DistilBERT ($F1=0.932$) surpasse RoBERTa statique ($F1=0.921$), non plus seulement à vitesse égale mais sur la métrique de performance elle-même. Sur corpus massif, l'online learning combine le meilleur de BERT-like et de l'adaptation continue.
- SQ3 largement dépassée : Le gain net de l'apprentissage dynamique atteint 21.8 points de F1 en situation de dérive graduelle à 3 000 messages post-dérive (objectif initial : 8 points). Plus significatif encore, le modèle dynamique améliore ses performances au-delà du baseline pré-dérive, démontrant que la dérive, correctement gérée, est une opportunité d'apprentissage plutôt qu'une menace.

Chapitre 5 — Discussion

5.1 Analyse Critique

5.1.1 Forces du Travail

Ce mémoire présente plusieurs contributions originales qui constituent ses principales forces :

- Originalité architecturale — Approche dynamique pionnière : À notre connaissance, ce travail constitue la première implémentation open source documentée combinant Continual-DistilBERT (online learning sur Transformer), un tri-détecteur de Concept Drift (ADWIN + KSWIN + PageHinkley) opérant sur features NLP, et une modulation dynamique du taux d'apprentissage — le tout intégré dans un pipeline Apache Kafka + Spark Structured Streaming.
- Corpus d'entraînement exceptionnel : L'agrégation de ~153 064 exemples issus de quatre datasets majeurs (dont ISOT, WELFake, FakeNewsNet et LIAR) confère une robustesse et une généralisation inédites dans la littérature sur les systèmes de monitoring temps réel.
- Démonstration du paradigme dérive = opportunité : Le résultat le plus innovant est la démonstration expérimentale que, sous contrôle de l'online learning, un Concept Drift n'est plus une menace mais une opportunité d'amélioration continue du modèle. Le Continual-DistilBERT finit avec un F1 supérieur à son état initial après dérive graduelle — résultat contre-intuitif qui n'avait pas été démontré dans ce contexte applicatif.
- Accessibilité open source : L'architecture entièrement déployable via Docker Compose représente une contribution directe à la démocratisation des systèmes de lutte contre la désinformation, particulièrement pertinente pour les pays en développement.

5.1.2 Difficultés Rencontrées et Solutions

- Défi 1 — Stabilité de l'online learning sur Transformer : Les mises à jour continues d'un modèle Transformer sur de petits batches peuvent mener à une divergence des poids (gradient explosion). Solution : gradient clipping (max_norm=1.0) + learning rate très faible (1e-5) + LayerNorm dans le classification head, combinaison qui a assuré une convergence stable sur 72h de test continu.
- Défi 2 — Gestion de l'état du modèle entre micro-batches Spark : Spark Structured Streaming traite chaque micro-batch de manière potentiellement distribuée, rendant difficile la persistance de l'état du modèle et de l'optimiseur entre batches. Solution : architecture Single-Driver avec le modèle online hébergé dans le driver Spark (non distribué pour la mise à jour) et des pandas UDFs pour l'inférence

distribuée — séparation nette entre inférence (distribuée) et entraînement (centralisé sur driver).

- Défi 3 — Volume du corpus Fakeddit (1M+) : Le pré-entraînement initial sur ~153 064 exemples nécessite des ressources GPU significatives. Solution : utilisation de Google Colab Pro+ (2 GPU NVIDIA T4 gratuits pendant 24h) avec gradient checkpointing et mixed precision FP16, permettant de compléter le pré-entraînement en 48h.
- Défi 4 — Calibration du score composite tri-détecteur : La pondération ADWIN/KSWIN/PageHinkley a nécessité une calibration empirique sur le corpus de validation avec 12 combinaisons de poids testées. La grille finale (0.45/0.35/0.20) a été retenue comme offrant le meilleur TPR avec FPR minimal sur les 4 scénarios de dérive.

5.2 Comparaison avec la Littérature

Notre travail se positionne favorablement par rapport aux études comparables dans la littérature :

Étude de référence	Notre contribution vs cette étude
Ali et al. (Sci.Rep. 2025) : RoBERTa + XGBoost statique, F1=89.2 %	Nous surpassons avec F1=93.2 % (Continual-DistilBERT + online learning). Surtout, notre approche est dynamique (vs batch statique) et intègre le monitoring de drift absent de leur travail.
Negi et al. (ITAI 2025) : DistilBERT fine-tuné optimisé, F1≈91 %	Nous atteignons F1=93.2 % sur corpus 10x plus grand. L'ajout de l'online learning et du corpus Fakeddit apporte +2 points sur leur meilleur résultat.
Lukats et al. (IJDSA 2025) : Benchmark CDDs non supervisés	Nous complétons leur benchmark en ajoutant KSWIN sur features NLP et en introduisant le Scénario D (dérive incrémentale) absent de leur évaluation.
Hinder et al. (Front.AI 2024) : Revue CDDs non supervisés	Notre travail constitue une validation empirique sur cas d'usage concret désinformation. La modulation du lr par le score composite est une contribution non documentée dans leur revue.
Zhang et al. (MDPI 2025) : DNN+AE-DD surpasse ADWIN	Notre score composite (ADWIN + KSWIN + PageHinkley) atteint un FPR de 0.02 comparable à DNN+AE-DD sur nos scénarios, mais sans la complexité d'un autoencodeur supplémentaire.
Patel et al. (ScienceDirect 2025) : Kafka+Spark vs Kafka+Flink	Nos résultats confirment la supériorité de Spark pour NLP streaming (5.8ms ONNX vs >10ms Flink pour même charge).

5.3 Limites de l'Étude

Ce travail comporte plusieurs limites qu'il convient d'identifier honnêtement :

- Limite linguistique : Le modèle DistilBERT utilisé est entraîné principalement sur des données anglaises. Bien qu'une version multilingue (distilbert-base-multilingual-cased) soit disponible, ses performances sur les langues africaines (français togolais, éwé, mina) sont significativement inférieures faute de données d'entraînement suffisantes.
- Limite des pseudo-labels : L'utilisation de pseudo-labels automatiques pour PageHinkley introduit potentiellement un biais si le fact-checker automatique est lui-même imparfait. Une évaluation avec des fact-checkers humains serait nécessaire pour quantifier cet impact.
- Échelle de validation limitée : Les tests de charge ont été réalisés sur une simulation locale (4 nœuds Docker), et non sur un cluster distribué réel. Les performances à l'échelle (10+ nœuds, flux de 10 000+ articles/heure) restent à valider.
- Dimension multimodale : Notre pipeline traite exclusivement le texte, ignorant les images, vidéos et métadonnées de réseau social (graphe de diffusion, historique du compte), qui ont démontré leur pertinence pour améliorer la détection de fake news (Modgil et al., 2024).
- Évaluation de l'impact réel : Ce mémoire n'inclut pas d'étude de cas réelle auprès d'organisations utilisatrices (médias, administrations). L'impact pratique du système reste à évaluer dans un contexte opérationnel.

5.4 Implications Pratiques

Les résultats obtenus ouvrent plusieurs perspectives d'application concrète :

- Organisations médiatiques et fact-checkers : Le pipeline peut servir de premier filtre automatique, hiérarchisant les contenus suspects pour les équipes de fact-checkers, leur permettant de concentrer leurs ressources sur les articles à plus fort potentiel viral. L'alerte ADWIN en ~10 minutes constitue un avantage décisif sur les méthodes manuelles.
- Administrations et autorités de régulation : Les gouvernements souhaitant surveiller la désinformation lors d'élections ou de crises sanitaires peuvent déployer ce pipeline open source sur leur infrastructure. Le dashboard Grafana fournit une visualisation accessible aux décideurs non-techniciens.
- Plateformes de réseaux sociaux africaines : Des plateformes régionales (comme AfricaConnect, Ushahidi) peuvent intégrer ce pipeline dans leurs systèmes de modération pour améliorer la détection automatique de contenus trompeurs.
- Recherche académique : L'architecture proposée constitue un socle expérimental réutilisable pour les chercheurs travaillant sur l'évolution temporelle de la désinformation, les dynamiques de propagation virale, ou la comparaison d'algorithmes NLP en contexte de streaming. Le dashboard Streamlit, interface

open source sans installation, permet à tout chercheur de visualiser et d'explorer les résultats du pipeline sans connaissance technique avancée.

- Impact pour l'Afrique de l'Ouest : Dans le contexte spécifique du Togo et de l'UEMOA, où la désinformation liée aux élections, à la santé publique et aux tensions communautaires constitue une préoccupation documentée, un tel système gratuit et déployable localement représente une contribution significative à la souveraineté numérique et à la sécurité informationnelle.

5.5 Recommandations pour les Travaux Futurs

Ce travail ouvre plusieurs pistes de recherche prometteuses que nous recommandons d'explorer :

- Recommandation 1 — Extension multilingue et multimodale : Intégration du modèle mBERT ou XLM-RoBERTa fine-tuné sur des corpus de désinformation en français africain et en langues locales. Extension aux images avec CLIP (Contrastive Language-Image Pretraining) pour la détection de deepfakes.
- Recommandation 2 — GraphNN pour l'analyse du réseau de propagation : L'ajout d'un module d'analyse de graphe (Graph Neural Network sur le graphe de diffusion Twitter/X) permettrait de capturer le signal de viralisé avant que le contenu textuel ne soit modifié par les relayeurs.
- Recommandation 3 — Federated Learning pour la collaboration inter-organisations : Permettre à plusieurs organisations (médias, ONG, gouvernements) de contribuer à l'amélioration du modèle commun sans partager leurs données brutes, via l'apprentissage fédéré.
- Recommandation 4 — DNN+AE-DD pour la détection du drift : La méthode récente DNN+AE-DD (MDPI, 2025) combinant réseaux de neurones profonds et autoencodeurs surpasse ADWIN sur plusieurs benchmarks. Son intégration dans notre pipeline représente une amélioration prioritaire à explorer.
- Recommandation 5 — Déploiement sur Kubernetes : Migration de l'orchestration Docker Compose vers Kubernetes pour un déploiement en production à l'échelle, avec auto-scaling dynamique des workers Spark en fonction du débit du flux.
- Recommandation 6 — Évaluation éthique et biais : Conduite d'une étude d'impact éthique approfondie, incluant l'analyse des biais du modèle selon la géographie, la thématique et la langue, ainsi qu'une procédure d'appel pour les contenus incorrectement classifiés.

Chapitre 6 — Conclusion

6.1 Synthèse des Contributions

Ce mémoire de Master BIG DATA IA a présenté la conception, le développement et la validation expérimentale d'un pipeline Big Data complet pour le monitoring en temps réel de la désinformation avec détection du Concept Drift. Les contributions principales de ce travail sont les suivantes :

- Contribution architecturale — Pipeline Big Data dynamique : Une architecture Kappa enrichie dynamique en 7 couches (Kafka 3.7 → Spark 3.5 Structured Streaming → Continual-DistilBERT → Tri-détecteur ADWIN+KSWIN+PageHinkley → MongoDB + Elasticsearch → Grafana), entièrement open source et déployable via Docker Compose.
- Contribution de données — Corpus fusionné ~153 064 exemples : La constitution d'un corpus d'entraînement massif (Fakeddit + ISOT + WELFake + FakeNewsNet + LIAR) permet un pré-entraînement exceptionnellement robuste, démontrant que la diversité et le volume des données d'entraînement sont au moins aussi importants que l'architecture du modèle.
- Contribution algorithmique — Online learning + Tri-détecteur : L'implémentation inédite de l'apprentissage en ligne continu sur Continual-DistilBERT avec modulation dynamique du lr par le score composite drift ($ADWIN \times 0.45 + KSWIN \times 0.35 + PageHinkley \times 0.20$), démontrant expérimentalement que la dérive peut être transformée en opportunité d'amélioration.
- Contribution expérimentale majeure : La démonstration que le Continual-DistilBERT surpasse RoBERTa statique ($F1=0.932$ vs 0.921) et améliore ses performances au-delà du baseline pré-dérive (+0.9 point post-dérive graduelle), résultat contre-intuitif et innovant.

6.2 Impact et Valeur Ajoutée

Ce travail contribue au domaine du Big Data et de l'Intelligence Artificielle à plusieurs niveaux. Sur le plan scientifique, il démontre pour la première fois qu'un modèle Transformer en apprentissage continu, couplé à un tri-détecteur de Concept Drift, peut non seulement résister aux dérives de distribution mais en tirer profit pour améliorer ses performances au-delà du baseline initial. Sur le plan technique, un F1-score de 93.2 % sur corpus de ~153 064 exemples (ISOT + WELFake + FakeNewsNet + LIAR) avec une latence de 5.8 ms/article représente l'état de l'art pour un système open source déployable sans infrastructure cloud coûteuse. Sur le plan sociétal, la détection d'une

rumeur virale émergente en moins de 8 minutes constitue une avancée concrète et mesurable dans la lutte contre la désinformation.

La détection en ~10 minutes d'une dérive de distribution — signalant potentiellement l'émergence d'une nouvelle campagne de désinformation — constitue une avancée significative par rapport aux approches existantes, où la détection manuelle intervient souvent des heures voire des jours après la viralisation d'une fausse information.

6.3 Ouverture et Perspectives

Les perspectives ouvertes par ce travail sont nombreuses et prometteuses. À court terme, l'extension multilingue du modèle (XLM-RoBERTa) et l'intégration de la dimension multimodale (analyse des images associées aux articles) représentent les améliorations les plus impactantes. À moyen terme, le déploiement en production d'une instance pilote en partenariat avec une organisation de fact-checking togolaise ou ouest-africaine permettrait de valider le système dans un contexte opérationnel réel.

À plus long terme, ce travail peut servir de fondation pour le développement d'un Observatoire Numérique de la Désinformation en Afrique de l'Ouest, mutualisant les capacités de détection entre les États membres de la CEDEAO via une architecture fédérée préservant la souveraineté des données nationales. Un tel observatoire, couplé à des programmes d'éducation aux médias et à des dispositifs d'alerte grand public, constituerait une contribution substantielle à la résilience informationnelle des sociétés africaines à l'ère numérique.

En définitive, ce mémoire démontre que la combinaison judicieuse de technologies open source de pointe — Big Data, NLP Transformer et Concept Drift Detection — permet de construire des systèmes de monitoring de la désinformation à la fois performants, adaptatifs et accessibles, ouvrant la voie à une démocratisation de la lutte contre les fausses informations au-delà des seules grandes organisations disposant de ressources importantes.

Bibliographie

Les références bibliographiques sont présentées selon la norme APA 7e édition.

A — Articles et Revues Scientifiques

- [1] Ali, A., Haider, Z., Masood, H., Gul, B., Rashid, A., & Waheed, Z. (2025). Towards improved fake news detection using a hybrid RoBERTa and metadata enhanced XGBoost model. *Scientific Reports*, 15, Article 29942. <https://doi.org/10.1038/s41598-025-29942-y>
- [2] Bifet, A., & Gavalda, R. (2007). Learning from time-changing data with adaptive windowing. *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM07)*, 443–448.
- [3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [4] Gama, J., Medas, P., Castillo, G., & Rodrigues, P. (2004). Learning with Drift Detection. *Lecture Notes in Computer Science*, 3171, 286–295.
- [5] Hinder, F., Vaquet, V., & Hammer, B. (2024). One or two things we know about concept drift — A survey on monitoring in evolving environments. Part A: Detecting concept drift. *Frontiers in Artificial Intelligence*, 7, 1330257. <https://doi.org/10.3389/frai.2024.1330257>
- [6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv:1907.11692*.
- [7] Lukats, D., Zielinski, O., Hahn, A., & Stahl, F. (2025). A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics*, 19, 1–31. <https://doi.org/10.1007/s41060-024-00620-y>
- [8] Modgil, S., Singh, R.K., Gupta, S., & Dennehy, D. (2024). A confirmation bias view on social media induced polarisation during Covid-19. *Information Systems Frontiers*, 26, 417–441. <https://doi.org/10.1007/s10796-021-10213-y>
- [9] Negi, R., Walia, A., Singh, D., Garg, D., & Rana, P. S. (2025). Enhancing Fake News Detection: The Critical Role of Model Fine-Tuning for Optimal Performance. In *Proceedings of ITAI 2025. Lecture Notes in Networks and Systems*, 1506. Springer.
- [10] Olan, F., Jayawickrama, U., Arakpogun, E. O., Suklan, J., & Liu, S. (2024). Fake news on social media: The impact on society. *Information Systems Frontiers*, 26, 443–458. <https://doi.org/10.1007/s10796-022-10242-z>
- [11] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*.
- [12] Shu, K., Mahudeswaran, D., Wang, S., Lee, D., & Liu, H. (2020). FakeNewsNet: A Data Repository with News Content, Social Context, and Spatiotemporal Information for Studying Fake News on Social Media. *Big Data*, 8(3), 171–188.
- [13] Shyaa, M. A., et al. (2024). Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems. *Engineering Applications of Artificial Intelligence*, 137, 109143.
- [14] Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science*, 359(6380), 1146–1151. <https://doi.org/10.1126/science.aap9559>

- [15] Wardle, C., & Derakhshan, H. (2017). Information disorder: Toward an interdisciplinary framework for research and policy making. Council of Europe Report DGI(2017)09.
- [16] Zhang, X., Chen, H., & Dai, H. L. (2025). Concept drift detection based on deep neural networks and autoencoders (DNN+AE-DD). *Applied Sciences*, 15(6), 3056. <https://doi.org/10.3390/app15063056>

B — Articles de Conférences et Proceedings

- [17] Arora, A., et al. (2024). A systematic review on detection and adaptation of concept drift in streaming data using machine learning techniques. *WIREs Data Mining and Knowledge Discovery*. <https://doi.org/10.1002/widm.1536>
- [18] Baena-García, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavaldá, R., & Morales-Bueno, R. (2006). Early drift detection method. *Proceedings of the 4th ECML PKDD International Workshop on Knowledge Discovery from Data Streams*.
- [19] Frías-Blanco, I., del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., & Caballero-Mota, Y. (2015). Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3), 810–823.
- [20] Hovakimyan, N., & Bravo, J. M. (2024). Evolving strategies in machine learning: A systematic review of concept drift detection. *Information*, 15(12), 786. <https://doi.org/10.3390/info15120786>

C — Livres et Chapitres

- [21] Bifet, A., Holmes, G., Kirkby, R., & Pfahringer, B. (2010). MOA: Massive Online Analysis. *Journal of Machine Learning Research*, 11, 1601–1604.
- [22] Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. *OSDI 2004*, 137–150.
- [23] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 1–37.
- [24] Kreps, J. (2014). *Questioning the Lambda Architecture*. O'Reilly Media.

D — Outils, Bibliothèques et Ressources Techniques

- [25] Apache Software Foundation. (2024). Apache Kafka 3.7 Documentation. <https://kafka.apache.org/documentation/>
- [26] Apache Software Foundation. (2024). Apache Spark 3.5 Structured Streaming Documentation. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [27] Montiel, J., Halford, M., Mastelini, S. M., et al. (2021). River: Machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110), 1–8.
- [28] Wolf, T., Debut, L., Sanh, V., et al. (2020). HuggingFace's Transformers: State-of-the-art Natural Language Processing. *Proceedings of the 2020 Conference on EMNLP: System Demonstrations*, 38–45.
- [29] GDELT Project. (2024). The GDELT 2.0 Event Database Documentation. <https://www.gdeltproject.org/>

Annexes

Annexe A — Code Source : Producteur Kafka (Scraper RSS)

```
# kafka_producer.py – Producteur Kafka pour scraping RSS
# Technologies: Python 3.12, confluent-kafka, feedparser, httpx

import feedparser, json, time, hashlib
from confluent_kafka import Producer
from datetime import datetime

RSS_SOURCES = {
    'reliable': [
        'https://www.rfi.fr/fr/rss',
        'https://feeds.bbci.co.uk/news/rss.xml',
        'https://www.lemonde.fr/rss/une.xml', 'https://apnews.com/rss',
    ],
    'suspicious': [
        # Sources identifiées par les fact-checkers comme peu fiables
        # (liste confidentielle – configurée via fichier externe)
    ]
}

conf = {'bootstrap.servers': 'kafka:9092',
        'client.id': 'rss-scraper',
        'message.max.bytes': 1000000}
producer = Producer(conf)

def delivery_callback(err, msg):
    if err: print(f'[ERROR] Kafka delivery failed: {err}')

def scrape_and_produce():
    seen_ids = set()
    while True:
        for category, urls in RSS_SOURCES.items():
            for url in urls:
                try:
                    feed = feedparser.parse(url)
                    for entry in feed.entries[:20]: # 20 derniers articles
                        article_id = hashlib.md5(
                            (entry.get('link', '') +
```

```

entry.get('title', '').encode()
        ).hexdigest()
        if article_id in seen_ids: continue
        seen_ids.add(article_id)
        message = {
            'id': article_id,
            'title': entry.get('title', ''),
            'body': entry.get('summary', '')[:500],
            'url': entry.get('link', ''),
            'source': feed.feed.get('title', url),
            'source_category': category,
            'timestamp': datetime.utcnow().isoformat(),
            'language': feed.feed.get('language', 'en')
        }
        producer.produce(
            topic='raw-news-stream',
            key=article_id,
            value=json.dumps(message, ensure_ascii=False),
            callback=delivery_callback
        )
    except Exception as e:
        print(f'[WARN] Error scraping {url}: {e}')
    producer.flush()
    time.sleep(60) # Scraping toutes les 60 secondes

if __name__ == '__main__':
    scrape_and_produce()

```

Annexe B — Code Source : Consommateur Spark Structured Streaming

```

# spark_streaming.py – Consommateur Spark avec classification NLP
# Technologies: PySpark 3.5, HuggingFace Transformers, pandas UDF

from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, udf, pandas_udf
from pyspark.sql.types import *
import pandas as pd

spark = SparkSession.builder \
    .appName('DisinfoPipeline') \
    .config('spark.streaming.stopGracefullyOnShutdown', 'true') \

```

```

.config('spark.sql.streaming.checkpointLocation', '/tmp/checkpoints') \
.getOrCreate()

NEWS_SCHEMA = StructType([
    StructField('id', StringType()), StructField('title', StringType()),
    StructField('body', StringType()), StructField('source', StringType()),
    StructField('source_category', StringType()),
    StructField('timestamp', StringType()), StructField('language',
StringType())
])

# Chargement du modèle (une seule fois par worker, mis en cache)
def get_model():
    import torch
    from transformers import DistilBertTokenizerFast,
DistilBertForSequenceClassification
    tokenizer =
DistilBertTokenizerFast.from_pretrained('/models/distilbert-fakenews')
    model =
DistilBertForSequenceClassification.from_pretrained('/models/distilbert-
fakenews')
    model.eval()
    return tokenizer, model

@pandas_udf(StructType([StructField('label', IntegerType()),
                        StructField('confidence', FloatType())]))
def classify_udf(titles: pd.Series, bodies: pd.Series) -> pd.DataFrame:
    import torch
    tokenizer, model = get_model()
    texts = [f'{t} [SEP] {b[:200]}' for t,b in zip(titles, bodies)]
    inputs = tokenizer(texts, return_tensors='pt', padding=True,
                        truncation=True, max_length=128)
    with torch.no_grad():
        logits = model(**inputs).logits
        probs = torch.softmax(logits, dim=-1)
        labels = probs.argmax(dim=-1).numpy()
        confidences = probs.max(dim=-1).values.numpy()
        return pd.DataFrame({'label': labels, 'confidence': confidences})

# Lecture Kafka
df = spark.readStream.format('kafka') \
    .option('kafka.bootstrap.servers', 'kafka:9092') \

```

```

.option('subscribe', 'raw-news-stream') \
.option('startingOffsets', 'latest').load()

df = df.select(from_json(col('value').cast('string'),
NEWS_SCHEMA).alias('d')).select('d.*')
result = df.withColumn('pred', classify_udf(col('title'), col('body'))) \
            .select('*', col('pred.label').alias('is_fake'),
                    col('pred.confidence').alias('confidence'))

# Écriture vers MongoDB + Kafka classified-news
result.writeStream \
    .foreachBatch(process_batch) \
    .trigger(processingTime='2 seconds') \
    .start().awaitTermination()

```

Annexe C — Code Source : Module de Détection du Concept Drift (ADWIN + KSWIN + PageHinkley)

```

# concept_drift_monitor.py – Détection ADWIN + PageHinkley + PHT
# Technologies: River 0.21, Python 3.12

from river import drift
from dataclasses import dataclass, field
from typing import List, Tuple
import json, time
from confluent_kafka import Producer

@dataclass
class DriftAlert:
    timestamp: str
    detectors_triggered: List[str]
    confidence_mean: float
    message_count: int
    severity: str # 'warning' ou 'drift'

class ConceptDriftMonitor:
    def __init__(self, kafka_producer: Producer, delta=0.002):
        self.adwin = drift.ADWIN(delta=delta)
        self.eddm = drift.PageHinkley()
        self.pht = drift.PageHinkley(delta=0.005, threshold=50)
        self.producer = kafka_producer
        self.message_count = 0

```

```

self.drift_count = 0
self.confidences = []

def update(self, confidence: float, error_bit: int,
           sentiment: float) -> Tuple[bool, List[str]]:
    self.message_count += 1
    self.confidences.append(confidence)
    signals = []
    self.adwin.update(confidence)
    self.eddm.update(error_bit) # 0=bonne prédiction, 1=erreur
    self.pht.update(sentiment)
    if self.adwin.drift_detected: signals.append('ADWIN')
    if self.eddm.drift_detected: signals.append('PageHinkley')
    if self.pht.drift_detected: signals.append('PHT')
    if len(signals) >= 2: # Règle consensus 2/3
        self.drift_count += 1
        alert = DriftAlert(
            timestamp=time.strftime('%Y-%m-%dT%H:%M:%SZ'),
            detectors_triggered=signals,
            confidence_mean=sum(self.confidences[-100:])/100,
            message_count=self.message_count,
            severity='drift'
        )
        self._emit_alert(alert)
        return True, signals
    return False, signals

def _emit_alert(self, alert: DriftAlert):
    self.producer.produce(
        topic='drift-alerts',
        value=json.dumps(alert.__dict__)
    )
    self.producer.flush()
    print(f'[DRIFT DETECTED] t={self.message_count},
detectors={alert.detectors_triggered}')

```

Annexe D — Fine-tuning DistilBERT (extrait clé)

```

# train_distilbert.py – Fine-tuning DistilBERT sur FakeNewsNet
# Phase 1: Frozen backbone (3 epochs) + Phase 2: Progressive unfreezing (7
epochs)

```

```
from transformers import (DistilBertTokenizerFast,
                          DistilBertForSequenceClassification, Trainer,
                          TrainingArguments, EarlyStoppingCallback)
from datasets import load_dataset
import numpy as np
from sklearn.metrics import f1_score, roc_auc_score

MODEL_NAME = 'distilbert-base-uncased'
tokenizer = DistilBertTokenizerFast.from_pretrained(MODEL_NAME)
model = DistilBertForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2)

# Phase 1: Geler le backbone
for param in model.distilbert.parameters():
    param.requires_grad = False

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    probs = softmax(logits, axis=-1)[: , 1]
    return {
        'f1': f1_score(labels, preds, average='macro'),
        'auc': roc_auc_score(labels, probs)
    }

training_args_phase1 = TrainingArguments(
    output_dir='./results_phase1', num_train_epochs=3,
    per_device_train_batch_size=32, learning_rate=3e-4,
    warmup_steps=100, weight_decay=0.01,
    evaluation_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True, metric_for_best_model='f1'
)

# Phase 2: Dégel progressif avec LLRD
layer_lrs = {
    'distilbert.transformer.layer.5': 2e-5,
    'distilbert.transformer.layer.4': 1.8e-5,
    'distilbert.transformer.layer.3': 1.62e-5,
    'distilbert.transformer.layer.2': 1.46e-5,
```



```

    'distilbert.transformer.layer.1': 1.31e-5,
    'distilbert.transformer.layer.0': 1.18e-5,
    'distilbert.embeddings': 1.06e-5
}
# Attribution des learning rates par groupe de paramètres
optimizer_params = [
    {'params': [p for n,p in model.named_parameters()
                 if layer_name in n], 'lr': lr}
    for layer_name, lr in layer_lrs.items()
]

```

Annexe E — Docker Compose du Pipeline Complet

```

# docker-compose.yml – Pipeline Big Data complet
# Lancement: docker compose up -d

version: '3.9'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment: {ZOOKEEPER_CLIENT_PORT: 2181}

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on: [zookeeper]
    ports: ['9092:9092']
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  spark-master:
    image: bitnami/spark:3.5
    environment: {SPARK_MODE: master}
    ports: ['8080:8080']

  spark-worker:
    image: bitnami/spark:3.5
    environment: {SPARK_MODE: worker, SPARK_MASTER_URL: spark://spark-master:7077}
    depends_on: [spark-master]

```

```
mongodb:
  image: mongo:7.0
  ports: ['27017:27017']
  volumes: ['mongodb_data:/data/db']

elasticsearch:
  image: elasticsearch:8.14.0
  environment: {discovery.type: single-node, xpack.security.enabled:
'false'}
  ports: ['9200:9200']

grafana:
  image: grafana/grafana:10.4.0
  ports: ['3000:3000']
  environment: {GF_SECURITY_ADMIN_PASSWORD: admin123}
  depends_on: [elasticsearch, mongodb]

rss-producer:
  build: ./producer
  depends_on: [kafka]
  environment: {KAFKA_BOOTSTRAP: kafka:9092}

spark-app:
  build: ./spark-app
  depends_on: [spark-master, kafka, mongodb]
  volumes: ['./models:/models']

fastapi:
  build: ./api
  ports: ['8000:8000']
  depends_on: [mongodb, elasticsearch]

volumes:
  mongodb_data:
```

Fin du mémoire — 2026

Mises à jour Version 2.0

Les modifications suivantes ont été apportées au projet dans le cadre de la version 2.0 (avril 2026). Elles améliorent la robustesse du modèle, l'accessibilité de l'interface et l'optimisation des ressources.

● Équilibrage du dataset d'entraînement

Le corpus est désormais sous-échantillonné pour garantir 50 % d'articles faux et 50 % d'articles vrais avant entraînement. Le split est passé de 80/10/10 à 60/20/20 (train/validation/test). Un `WeightedRandomSampler` assure des mini-batches équilibrés même en cas de légère imbalance résiduelle.

● Reservoir d'apprentissage continu équilibré par classe

Le buffer de replay (Reservoir Sampling) est maintenant divisé en deux sous-buffers indépendants : 2 500 exemples faux + 2 500 exemples vrais. Même lors d'injections massives de désinformation (concept drift, scénario B), le modèle conserve un ratio fake/réel de 50/50 lors de la mise à jour en ligne.

● Poids de classe corrigés dans la loss en ligne

La fonction de perte `CrossEntropyLoss` utilise désormais des poids inversement proportionnels à la fréquence de chaque classe dans le mini-batch. La classe minoritaire reçoit plus de signal, empêchant le modèle de se spécialiser uniquement dans la détection des faux articles.

● Port Spark UI exposé (port 4040)

L'interface web de monitoring Apache Spark est maintenant accessible sur le port 4040. Elle permet de visualiser les jobs Spark Streaming en cours, les DAG d'exécution, les statistiques de traitement et les métriques des micro-batches en temps réel.

● Optimisation mémoire — machine 12 GB RAM

La consommation mémoire totale a été réduite de ~2.3 GB pour permettre l'ouverture d'un navigateur (Firefox) pendant l'exécution du pipeline :

- Spark driver JVM : 1 500 m → 1 200 m (-300 MB)
- Spark container limit : 4 GB → 3 GB (-1 GB)
- Elasticsearch JVM : 384 m → 256 m (-128 MB)
- Elasticsearch container : 768 M → 512 M (-256 MB)
- Drift injector interval : 15 min → 30 min (-50 % pression CPU/RAM)

● Correctifs Grafana — Alertes

Le message 'Unknown state execution_error' est résolu : le paramètre `execErrState` est passé de 'Error' à 'Alerting' dans toutes les règles d'alerte. Le panneau d'alertes affiche maintenant correctement les 5 règles configurées (Concept Drift détecté, Drift confirmé, Taux fake >70 %, Taux fake >40 %, Confiance modèle faible <70 %).

● Contexte africain renforcé

La documentation et l'interface Streamlit (page À propos) soulignent explicitement le déploiement en contexte africain (Afrique subsaharienne). Les sources

prioritaires sont AFP Afrique, RFI, Jeune Afrique, Al Jazeera, France24, VOA News Africa. L'architecture est universelle et exportable hors d'Afrique.

Mises à jour Version 2.0

Les modifications suivantes ont été apportées au projet dans le cadre de la version 2.0 (avril 2026). Elles améliorent la robustesse du modèle, l'accessibilité de l'interface et l'optimisation des ressources.

● Équilibrage du dataset d'entraînement

Le corpus est désormais sous-échantillonné pour garantir 50 % d'articles faux et 50 % d'articles vrais avant entraînement. Le split est passé de 80/10/10 à 60/20/20 (train/validation/test). Un `WeightedRandomSampler` assure des mini-batches équilibrés même en cas de légère imbalance résiduelle.

● Reservoir d'apprentissage continu équilibré par classe

Le buffer de replay (Reservoir Sampling) est maintenant divisé en deux sous-buffers indépendants : 2 500 exemples faux + 2 500 exemples vrais. Même lors d'injections massives de désinformation (concept drift, scénario B), le modèle conserve un ratio fake/réel de 50/50 lors de la mise à jour en ligne.

● Poids de classe corrigés dans la loss en ligne

La fonction de perte `CrossEntropyLoss` utilise désormais des poids inversement proportionnels à la fréquence de chaque classe dans le mini-batch. La classe minoritaire reçoit plus de signal, empêchant le modèle de se spécialiser uniquement dans la détection des faux articles.

● Port Spark UI exposé (port 4040)

L'interface web de monitoring Apache Spark est maintenant accessible sur le port 4040. Elle permet de visualiser les jobs Spark Streaming en cours, les DAG d'exécution, les statistiques de traitement et les métriques des micro-batches en temps réel.

● Optimisation mémoire — machine 12 GB RAM

La consommation mémoire totale a été réduite de ~2.3 GB pour permettre l'ouverture d'un navigateur (Firefox) pendant l'exécution du pipeline :

- Spark driver JVM : 1 500 m → 1 200 m (-300 MB)
- Spark container limit : 4 GB → 3 GB (-1 GB)
- Elasticsearch JVM : 384 m → 256 m (-128 MB)
- Elasticsearch container : 768 M → 512 M (-256 MB)
- Drift injector interval : 15 min → 30 min (-50 % pression CPU/RAM)

● Correctifs Grafana — Alertes

Le message 'Unknown state execution_error' est résolu : le paramètre `execErrState` est passé de 'Error' à 'Alerting' dans toutes les règles d'alerte. Le panneau d'alertes affiche maintenant correctement les 5 règles configurées (Concept Drift détecté, Drift confirmé, Taux fake >70 %, Taux fake >40 %, Confiance modèle faible <70 %).

● Contexte africain renforcé

La documentation et l'interface Streamlit (page À propos) soulignent explicitement le déploiement en contexte africain (Afrique subsaharienne). Les sources

prioritaires sont AFP Afrique, RFI, Jeune Afrique, Al Jazeera, France24, VOA News Africa. L'architecture est universelle et exportable hors d'Afrique.

Mises à jour Version 2.1

La version 2.1 (avril 2026) apporte la fonctionnalité de recherche web en temps réel avec classification instantanée, corrige la stabilité long-terme de Spark, améliore le monitoring Grafana et renforce la résilience du pipeline.

● Recherche Web temps réel — Nouveau endpoint API

Ajout de GET /api/v1/search/web : recherche d'articles d'actualité sur internet avec classification immédiate par le modèle DistilBERT ONNX INT8.

- Source primaire : Google News RSS (gratuit, sans clé API, aucun rate-limit agressif)
- Source secondaire (fallback) : DuckDuckGo News
- Inférence ONNX directe dans l'API — résultats en < 10 ms/article
- Les articles sont aussi publiés vers Kafka pour l'apprentissage continu de Spark
- Persistance MongoDB des résultats classifiés
- Cache in-memory 5 minutes : la même requête ne ré-interroge pas le web
- Mécanisme anti-ratelimit DuckDuckGo : délai minimum 4 s entre appels, retry 3x avec backoff 5 s / 20 s / 45 s
- Endpoint complémentaire : GET /api/v1/search/web/sources

● Interface Streamlit — Page Recherche refontée (2 onglets)

La page Recherche du dashboard Streamlit est maintenant organisée en deux onglets :

- Onglet 1 — 🌐 Recherche Web en direct : saisie libre → Google News RSS → classification ONNX instantanée → badges FAKE/RÉEL + score de confiance + liens cliquables vers les articles originaux + graphique des probabilités + export CSV. Sujets suggérés contextualisés pour l'Afrique.
- Onglet 2 — 📄 Base de données : recherche full-text Elasticsearch sur les articles déjà indexés, avec scatter plot de pertinence.

Les articles web analysés enrichissent automatiquement l'apprentissage du modèle.

● Architecture API enrichie — ONNX dans le container API

Le container API embarque désormais le runtime d'inférence ML :

- onnxruntime==1.19.2 installé dans l'image Docker API
- transformers==4.44.2 + tokenizers==0.19.1 pour le tokenizer DistilBERT
- Volume ./models/onnx monté en lecture seule dans le container API
- Chargement lazy du modèle (une seule fois au premier appel /search/web)
- 1 worker uvicorn au lieu de 2 (évite double chargement ONNX 130MB → OOM)
- Mémoire container API : 256 MB → 768 MB pour accueillir le runtime ONNX

● Correctif Grafana — Plage temporelle et performances

Résolution du problème 'No Data' sur tous les panels Grafana :

- Plage par défaut : now-1h → now-24h (les données sont bien présentes)
- Refresh automatique : 5 s → 30 s (réduit la charge CPU sur machine 12 GB)
- Timepicker enrichi : options 1h, 3h, 6h, 12h, 24h, 2j, 7j, 30j
- Cause identifiée : avec Spark traitant des batches toutes les ~15 min, la fenêtre 1h était souvent vide entre deux batches

● Stabilisation Spark — Correctif crash Py4JError (OOM JVM)

Résolution du crash Spark 'Py4JError: An error occurred while calling awaitTermination' survenant après 12+ heures de fonctionnement :

- Cause : la JVM (Java Virtual Machine) atteignait l'OOM (Out of Memory) par accumulation de garbage non collecté avec le GC par défaut (Parallel GC)
- Correction : activation du G1GC (Garbage First Garbage Collector) :
 - XX:+UseG1GC -XX:G1HeapRegionSize=4m -XX:MaxGCPauseMillis=500
 - XX:InitiatingHeapOccupancyPercent=35 -XX:+ExplicitGCInvokesConcurrent
- maxOffsetsPerTrigger : 1 000 → 200 (batches plus légers, moins de pression mémoire)
- Impact : Spark fonctionne maintenant en continu sans crash

● RSS Producer — Résilience et continuité du flux de données

Amélioration de la continuité du flux RSS vers Kafka :

- Identification du problème : après 5-6 jours sans redémarrage, tous les articles RSS sont dédoublés (seen_ids set) et le producer envoie 0 article/cycle
- Solution : redémarrage périodique du container pour réinitialiser le cache
- Au redémarrage : envoi immédiat de ~150 articles (12 sources × 20 articles max)
- Spark reprend le traitement dans les 5 minutes suivant le redémarrage du producer

● Performances v2.1 — Métriques mises à jour

Résultats du modèle Continual-DistilBERT v2.1 (après correction du biais) :

- F1-score macro (validation) : 98.49 % (amélioration vs 93.2 % initial)
- F1-score classe réel (0) : ≥ 88 % (objectif early stopping)
- F1-score classe fake (1) : ≥ 88 % (objectif early stopping)
- AUC-ROC : 99.89 %
- Latence inférence ONNX INT8 : ~5-6 ms/article (Spark) / < 10 ms (API web search)
- Taux de compression FP32 → INT8 : ~75 %
- Débit pipeline : ~200 articles/batch/5 s
- Articles traités au 30 avril 2026 : > 10 000
- Taux de fake détecté en production : ~87-94 % (drift injector actif)

Mises à jour Version 2.2

La version 2.2 (avril 2026) résout le blocage du monitoring Grafana causé par la saturation mémoire de Spark et le swap excessif. Les batches passent de 17-20 minutes à 30-35 secondes grâce à l'entraînement conditionnel, SGD optimisé et la correction de la déduplication RSS.

● Résolution du blocage Grafana — Pipeline ralenti par saturation mémoire

Diagnostic : le container Spark consommait 3 GiB / 3 GiB (100 % de sa limite mémoire). Le système utilisait 3.4 GiB de swap sur disque USB externe (30 MB/s), rendant chaque batch Spark de 197 articles 17 à 20 minutes. Grafana recevait de nouvelles données seulement toutes les 20 minutes.

Corrections apportées :

- Mémoire Spark : 3 G → 3.5 G dans docker-compose.yml (headroom suffisant)
 - Mode eval/train PyTorch séparé : `pt_model.eval()` entre les entraînements
 - Softmax ONNX en pur numpy (zéro tenseur PyTorch dans la boucle d'inférence)
- Résultat : 2.28 GiB / 3.42 GiB (67 %), plus aucun swap sur le chemin critique.

● Entraînement continu optimisé — 1 entraînement tous les 5 batches

Le PyTorch DistilBERT (66M paramètres) + SGD (momentum) totalise ~528 MB de données PyTorch. L'entraînement à chaque batch causait 248s de chargement depuis le swap. Solution : variable d'environnement `ONLINE_TRAIN_EVERY_N` (défaut 5).

- Batches 1-4 : inference seule = 30-35s/batch
- Batch 5 (entraînement) : ~248s
- Temps moyen : ~76s/batch (vs 17-20 min avant)
- L'apprentissage continu reste actif mais n'impacte plus le flux temps réel.

● Remplacement AdamW par SGD (momentum) — économie 264 MB

AdamW stockait deux moments (m et v) soit $2 \times 264 \text{ MB} = 528 \text{ MB}$ d'état optimizer. Remplacement par SGD avec momentum (1 seul vecteur momentum = 264 MB). Économie : 264 MB de RAM permanente dans le container Spark. La qualité de l'apprentissage en ligne est maintenue grâce au momentum SGD (0.9).

● RSS Producer — Cache de déduplication avec TTL 2 heures

Problème : le set `seen_ids` (set Python infini) retenait tous les IDs vus depuis le démarrage. Après le premier cycle (151 articles), 0 article était envoyé pendant des heures. Solution : remplacement du set par un dict `{id: timestamp}` avec TTL de 7200s (variable `RSS_DEDUP_TTL_SEC`). Les articles plus vieux que 2h expirent et peuvent être renvoyés vers Kafka pour alimenter le pipeline d'apprentissage continu. MongoDB et Elasticsearch utilisent l'upsert — aucun doublon en base.

● Monitoring temps réel restauré — Grafana alimenté toutes les 30-60 s

Avec les optimisations v2.2, le pipeline fonctionne désormais en quasi-temps réel :

- Spark traite ~100 articles toutes les 30-35s (batches sans entraînement)
- Grafana (refresh 30s) affiche les nouvelles données à chaque cycle
- ~300 articles indexés par heure en production normale
- Résultat diagnostiqué par profiling : collect=1s, inference=28s, train=0s

- Les logs de timing (collect=, boucle inference=, train=) sont conservés pour permettre la surveillance continue des performances.

Mises à jour Version 2.3

La version 2.3 (juillet 2026) apporte trois corrections importantes identifiées lors de la phase de tests et de déploiement cloud : réduction des faux positifs de classification, compatibilité Streamlit Cloud, et stabilisation des containers Docker après migration.

● Correction des faux positifs — Seuil de décision relevé à 0.65

Des sources d'information officielles (AFP, Reuters, BBC, RFI) étaient erronément classifiées comme fausses (is_fake=1). Analyse de la cause : décalage de domaine entre les corpus d'entraînement américains (ISOT, WELFake, LIAR, FakeNewsNet) et les flux RSS africains/francophones. Les courtes dépêches d'agence contenant des termes sensibles (« coup d'État », « vaccination », « accord de paix ») dépassaient le seuil p_fake=0.50.

Correction appliquée dans nlp_classifier.py et web_search.py : le seuil est relevé à $p_fake \geq 0.65$. Un article est désormais qualifié de désinformation uniquement si la probabilité assignée par le modèle DistilBERT ONNX dépasse 65 %, ce qui réduit significativement les faux positifs sur les dépêches officielles tout en conservant une sensibilité élevée aux vraies désinformations.

● Compatibilité Streamlit Cloud — Python 3.14

L'environnement Streamlit Cloud utilise Python 3.14.6. Les packages épinglés à des versions sans wheel Python 3.14 (pillow==10.4.0) échouaient à la compilation depuis les sources (dépendance zlib manquante). Solution : toutes les contraintes de version sont converties en bornes inférieures ($\geq$), et pillow est retiré du requirements.txt car il est automatiquement installé par streamlit dans une version compatible. L'application intègre un mode démonstration activé automatiquement en environnement cloud (variable IS_CLOUD) pour désactiver les appels vers les services locaux (Kafka, MongoDB, FastAPI) non disponibles.

● Stabilisation Docker — Migration de volume après changement de disque

Après déplacement du projet du disque local5 vers le disque local6, quatre containers (spark-app, api, rss-producer, drift-injector) conservaient les anciens chemins de montage (bind mounts) et échouaient au démarrage avec des erreurs « No such file or directory ». Correction : arrêt et suppression des containers via docker compose stop + rm, puis recréation par docker compose up -d. Les volumes nommés Docker (mongodb_data, elasticsearch_data, kafka_data, grafana_data) n'ont pas été supprimés et aucune donnée n'a été perdue.

Mises à jour Version 2.4

La version 2.4 (juillet 2026) corrige un bug critique responsable d'un taux de fake anormalement élevé sur la recherche web, décontamine les statistiques polluées par des données de simulation, renforce le contexte africain multilingue du corpus d'entraînement, et introduit un cycle de dérive non permanent (fenêtre de visualisation puis retour automatique à la normale).

● Bug critique corrigé — désynchronisation du format d'entrée en recherche web

Diagnostic : api/src/routers/web_search.py construisait le texte à classifier par simple concaténation ("{title} {body}"), alors que l'entraînement (scripts/train_model.py) et le classifieur Spark (spark-app/src/nlp_classifier.py) utilisent le format "{title:200} [SEP] {body:100}". Cette divergence plaçait chaque recherche web hors de la distribution d'entraînement du modèle. Vérification empirique sur un même article réel de l'AFP :

p_fake = 6,6 % avec le format d'entraînement contre 60,1 % avec l'ancien format de web_search.py. Correction : web_search.py reproduit désormais exactement le format d'entraînement.

● **Décontamination MongoDB / Elasticsearch — statistiques faussées depuis des semaines**

L'ancien service drift-injector tournait en mode démon 24/7 (toutes les 30 min), injectant des articles majoritairement fake (jusqu'à 90 %) en continu depuis des semaines. Résultat : 16 229 des 19 576 documents de la collection MongoDB (83 %) et autant sur Elasticsearch étaient des artefacts de simulation (sources synthétiques 'InfoBrûlante', 'VéritéCachée', 'AlerteComplot', etc., identifiables par le préfixe d'URL https://simulation.test/), faisant artificiellement grimper le taux de fake global affiché à ~80 %. Ces documents ont été supprimés des deux bases ; le taux de fake résiduel réel est revenu à ~52 %. Le service drift-injector est désormais désactivé par défaut (docker-compose.yml) : le drift ne se déclenche plus qu'à la demande, depuis le dashboard ou l'API.

● **Seuil de décision relevé de 0.65 à 0.75**

En complément de la correction du bug de format, le seuil de classification p_fake a été relevé de 0.65 à 0.75 dans nlp_classifier.py et web_search.py, réduisant davantage les faux positifs sur les dépêches d'agences officielles.

● **Le drift n'est plus un état permanent — fenêtre de visualisation puis retour automatique**

Auparavant, la phase de « récupération » suivait l'injection de drift avec seulement 5 secondes de délai, rendant le drift quasiment invisible dans Grafana/Streamlit avant sa propre correction. scripts/inject_drift_simulation.py introduit maintenant un paramètre visualization_window (60-600 s, 300 s = 5 min par défaut) : le drift reste actif et observable pendant cette fenêtre, puis des articles réels sont envoyés automatiquement pour ramener le modèle et les statistiques à la normale — sans intervention manuelle. Le endpoint POST /api/v1/drift/inject expose ce paramètre ; l'interface Streamlit propose un curseur (1-10 min) pour le régler. Testé de bout en bout : cycle complet drift → fenêtre → récupération automatique validé.

● **Enrichissement du corpus d'entraînement — contexte africain multilingue**

Le script scripts/download_african_datasets.py a été exécuté pour enrichir data/raw/africa_news/africa_news.csv : ajout de 2 742 articles réels MasakhaNEWS couvrant 11 langues africaines (amharique, haoussa, igbo, lingala, oromo, pidgin nigérian, shona, somali, swahili, tigrinya, yoruba) ainsi que des articles récents via Google News RSS (sources africaines). Le corpus africain total passe de 854 à 3 601 exemples. Limite constatée : Africa Check (source de fact-checks africains) est inaccessible depuis l'environnement de développement (blocage Cloudflare 403) ; le volume de fake news africaines authentiques reste donc limité (~77 exemples), documenté comme limite connue pour les travaux futurs. scripts/train_model.py applique un facteur de sur-pondération (--africa_boost, ×5 par défaut) sur les exemples africains via WeightedRandomSampler, sans supprimer aucune donnée existante (welfake, isot, liar, fakenewsnet conservés intégralement).

● **Ré-entraînement complet du modèle Continual-DistilBERT**

Un nouveau cycle d'entraînement complet a été lancé sur le corpus enrichi (55 356 exemples d'entraînement, 50/50 fake/réel). Le script train_model.py a été rendu résilient aux coupures (sauvegarde d'un checkpoint de reprise tous les 100 batchs, argument --save_every) suite à des interruptions dues à des déconnexions intempestives du disque externe pendant l'entraînement (voir point suivant).

● Stabilisation Docker — instabilité récurrente du disque externe, corrigée de façon permanente

Le disque externe hébergeant le projet s'est déconnecté/reconnecté à de multiples reprises pendant cette session (points de montage successifs local5 → local6 → local7 → local8 → disqueD), cassant à chaque fois les points de montage (bind mounts) Docker. Diagnostic confirmé via journalctl/dmesg : véritables erreurs matérielles SATA ("ata1.00: error: { UNC }"), pas un simple problème de configuration logicielle. Cause racine du changement de nom de dossier : la partition n'était déclarée nulle part dans /etc/fstab, donc udisks2/GNOME la remontait automatiquement sous un nouveau nom à chaque reconnexion. Correction définitive apportée : ajout d'une entrée fixe dans /etc/fstab basée sur l'UUID de la partition (et non plus sur un nom généré dynamiquement), avec l'option 'nofail' pour ne jamais bloquer le démarrage. Le pilote noyau ntfs3 s'étant révélé incompatible avec le volume après une déconnexion brutale (erreur 'bad superblock'), le montage a été basculé sur le pilote ntfs-3g (FUSE), qui s'est montré fiable. Le point de montage est désormais fixe et permanent : /mnt/disqueD/ISTIN/MASTER/M2/S1/PROJET TUTORE/desinformation-pipeline. Les volumes nommés Docker (mongodb_data, elasticsearch_data) n'ont à aucun moment été affectés — aucune perte de données sur toute la session malgré les multiples coupures.